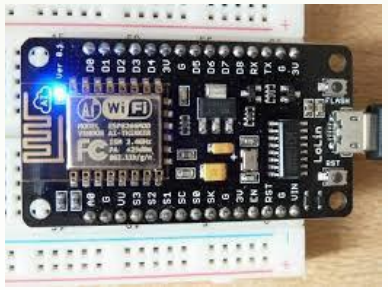
Chapitre prise en charge ESP32/ESP8266 sur IDE Arduino

1.1 Circuit ESP8266



ESP-01

2x GPIO*
512kB Flash



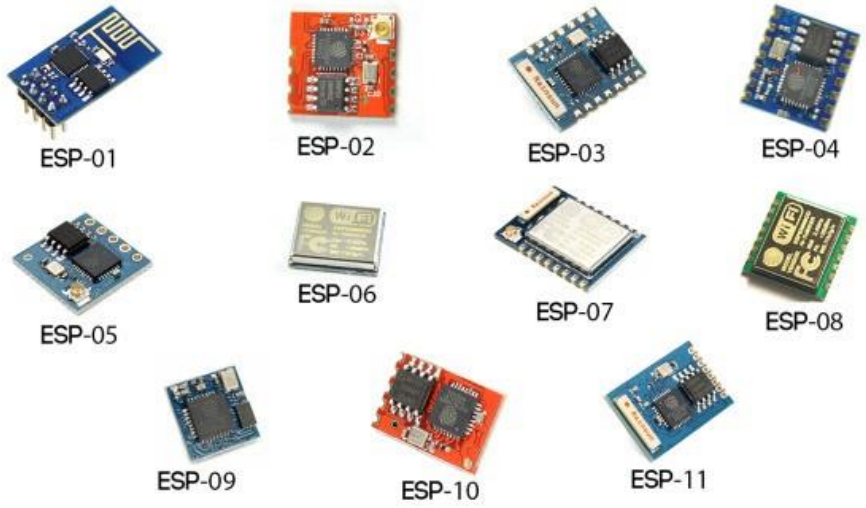
NodeMCU

11x GPIO
USB



WeMos D1

11x GPIO*
USB
Small Form Factor
4MB Flash **



*Le chip a 12 GPIO (+
6 utilisés par la mémoire)

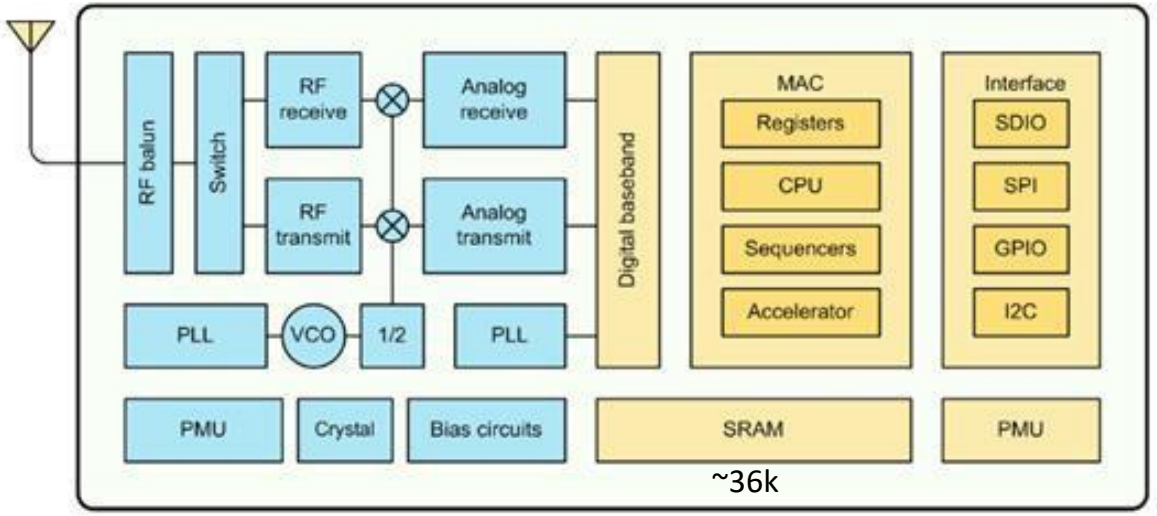
** Max 16MBits Flash

1.2 Circuit ESP8266



- Espressif Systems (Shanghai) Pte. Ltd.
  - 32 pin Ultra Low Power
  - WIFI 2.4GHz b,g,n WPA/WPA2
  - 80/160Mhz (pll)
  - 32 bits RTOS inclus (pas visible)
  - 80% de la puissance disponible (non wifi)

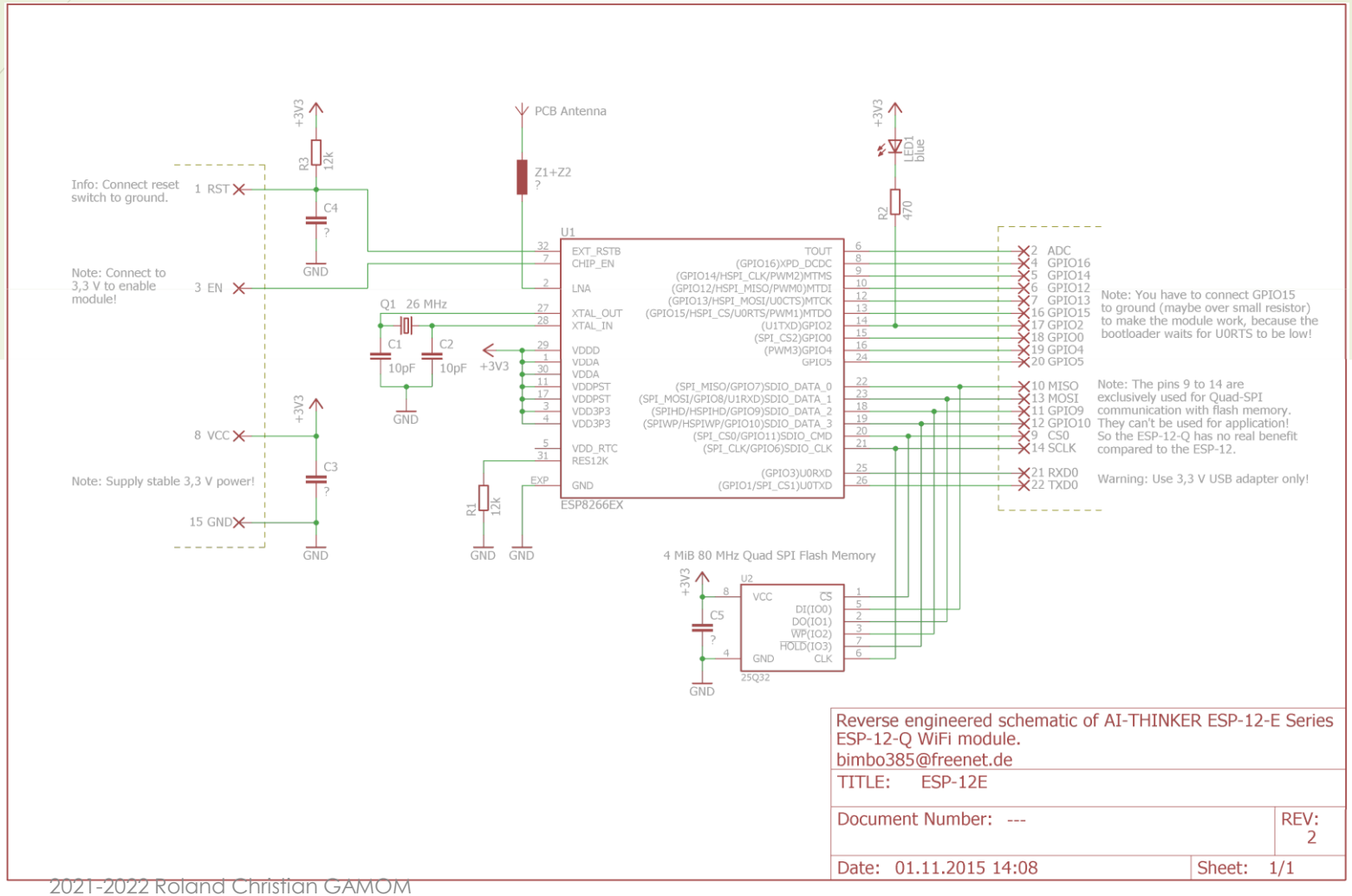
- Home Appliances
- Home Automation
- Smart Plug and lights
- Mesh Network
- Industrial Wireless Control
- Baby Monitors
- IP Cameras
- Sensor Networks
- Wearable Electronics



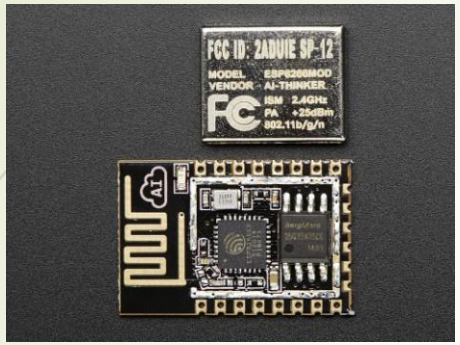
- Flash externe: 512b-16Mb

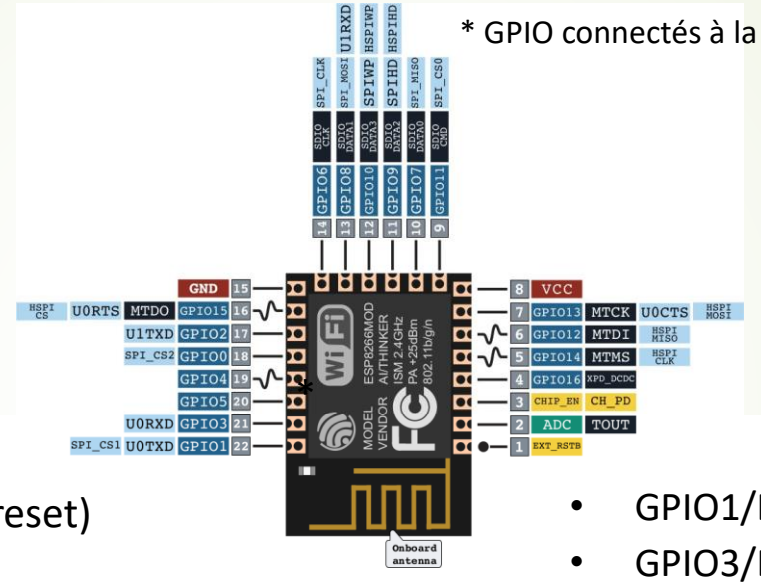
ARM Core: Tensilica LX106

ESP8266 Schéma



1.3 connexion de ESP8266



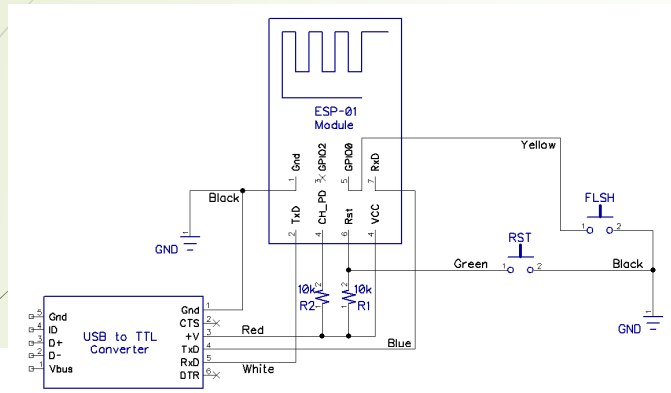


- Reset (High=run, Low=reset)
- ADC
- CH_PD, Enable (High=on, Low=off)
- GPIO16/D0/USER/WEAK
- GPIO14/D5/SPI-SCK
- GPIO12/D6/MISO
- GPIO13/D7/MOSI

- GPIO1/D10 TX0
- GPIO3/D9 RX0
- GPIO5 /D1/SCL(I2C)
- GPIO4 /D2/SDA(I2C)
- GPIO0/D3/FLASH
- GPIO2/D4
- GPIO15/D8/TX2/SPI-CS

Si 6 pins en bas, elles sont connectées au SPI interne / en parallèle sur la mémoire

ESP8266 boot mode



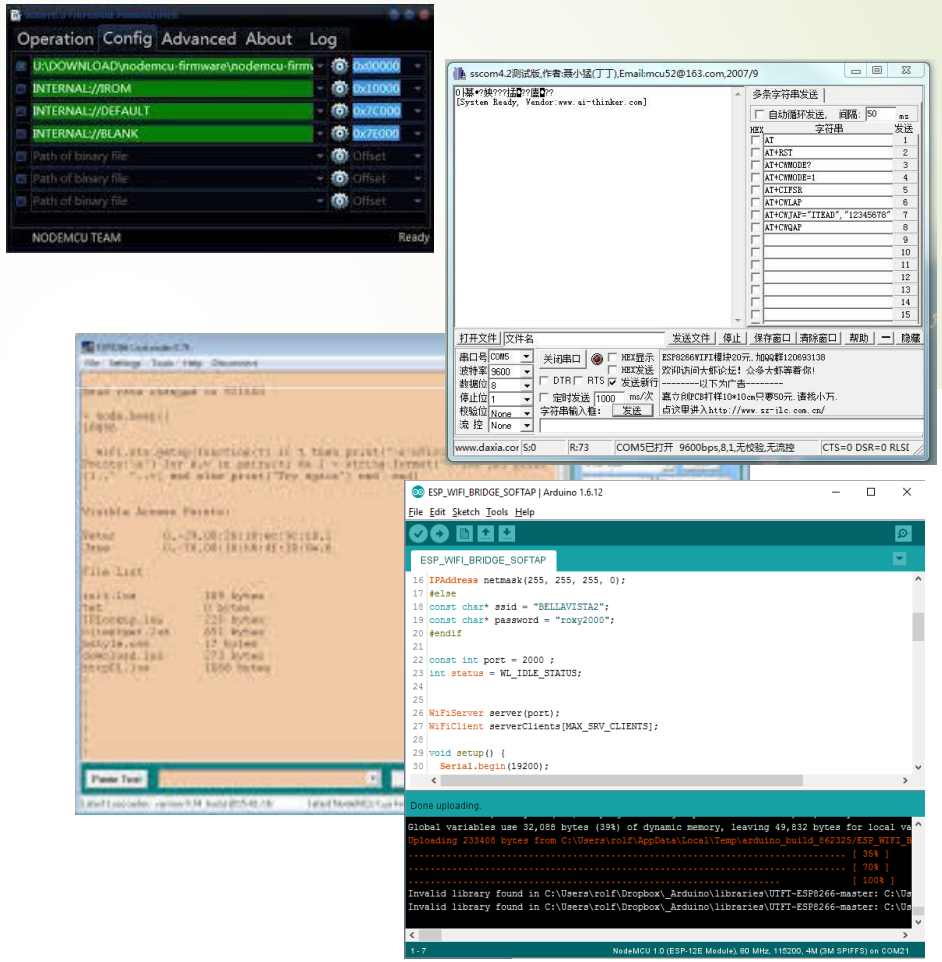
ESP-01 Connection Diagram

	GPIO 0	GPIO 2	GPIO 15
UART Download Mode (Programming)	0	1	0
Flash Startup (Normal)	1	1	0
SD-Card Boot	0	0	1

- Pour le mode normal,
  - GPIO15 est tenu à « 0 » par une résistance de 10k
  - GPIO 0 et GPIO 2 sont tenus à « 1 » par 10kOhm
- Mode Flash(par UART) GPIO0 est mis à 0 (Bouton,…)
- Seul avec une carte SD, GPIO2 doit être mise à 0 au démarrage (boot from SD card)

2. Logiciel de programmation

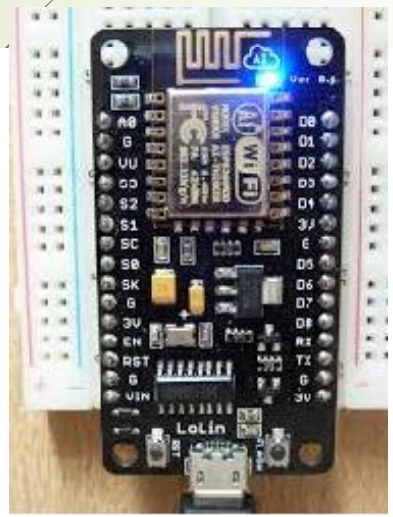
- Binary loader
- C code (linux)*
- Commandes AT
- Langage LUA**
- C code (Arduino)



* a charger avec « binary loader »
**acronyme « love you always »

3. Programmer ESP avec IDE Arduino

- Un plug-in permet de faire l'extension Arduino-ESP8266, il suffit d'ajouter:
  http://arduino.esp8266.com/stable/package_esp8266com_index.json (40MB)

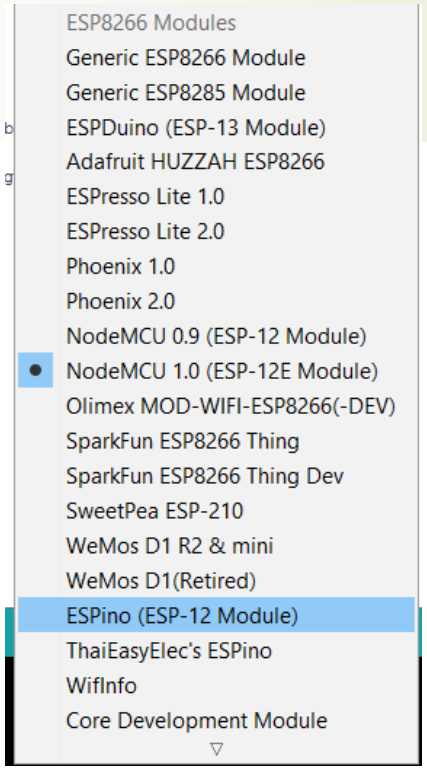


Débuter avec NodeMCU
-Grille 2.54
-Régulateur plus puissant
-Interface USB

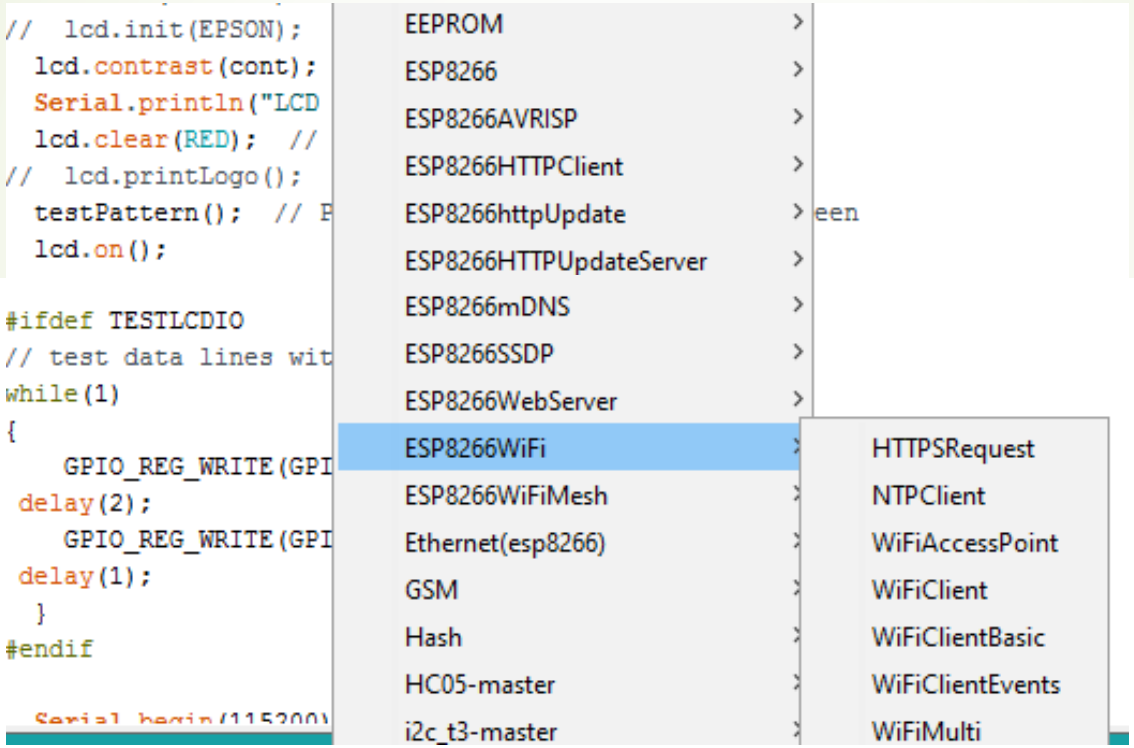
•De nombreuses **librairies** peuvent être ajoutée dans Arduino permettant le développement à partir d'exemples

•Avec l'extension, les cartes sont alors visibles **dans tools** sous Arduino

•Moyennant un interface USB (Node MCU/D1 ou avec FTDI) on peut alors flasher le ESP directement depuis l'environnement Arduino



3.1 Exemples Arduino



4. Modes sans fil (WIFI) ??
AP, SoftAP, STA

I2C/I2S
SPI
UART
GPIO

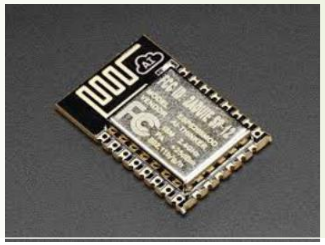
Niveau
Connexion (Wifi)

```
const char* ssid = « MICRONET";
const char* password = « 1234";
WiFi.begin(ssid, pass);
```



STA

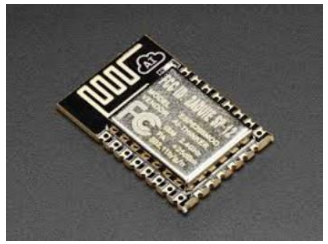




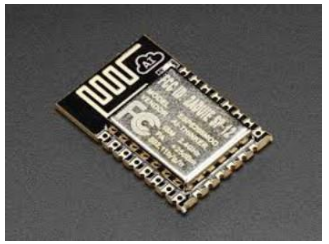
Niveau Protocol
(HTML, TCP, UDP)

```
Udp.begin(inPort)
Udp.read(buf,size);
....
```

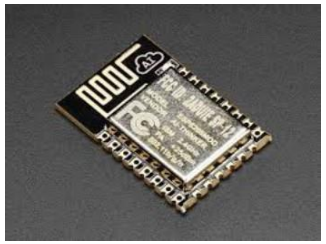
AP(server)



SOFTAP



Client



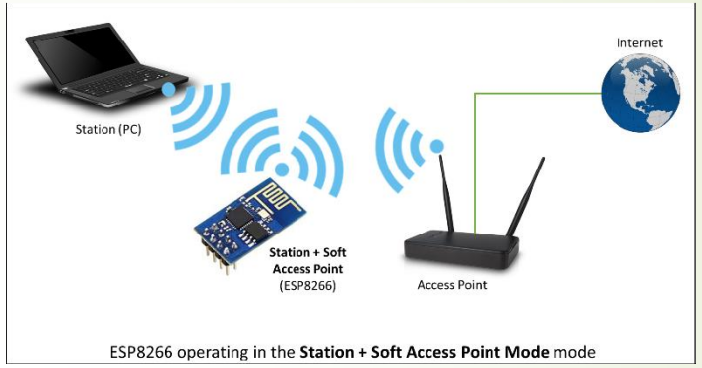
STA

Niveau Périphérie
I2C/I2S
SPI
UART
GPIO

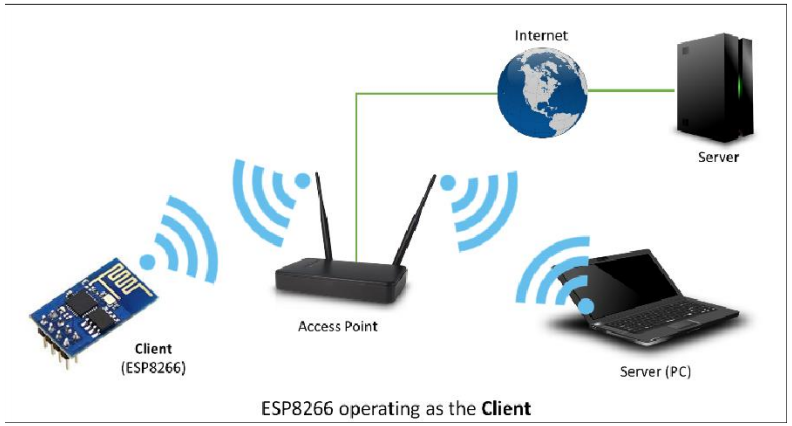
4.1Modes WIFI



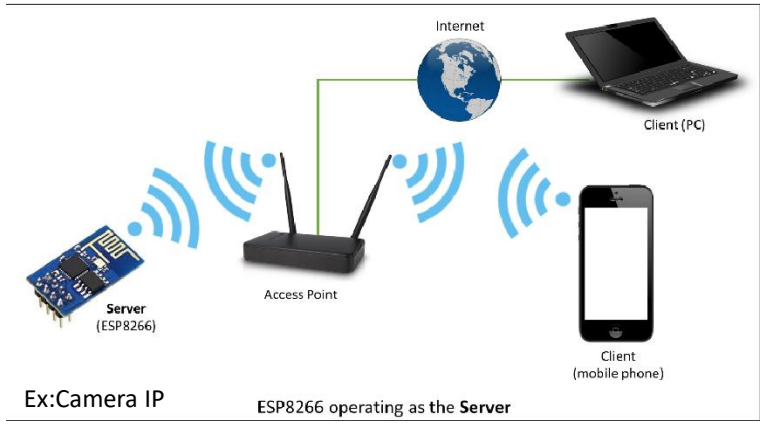
Connexion point-point sans PWD Ex: robot télécommandé



Connexion point-point sans PWD puis Station sur AP



Connexion sur AP, puis requête du réseau.
Ex. Horloge/Display Météo



Connexion sur AP et répond aux requêtes du réseau
Les clients de l'AP peuvent se connecter sur le server

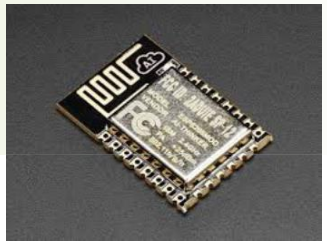
https://www.gitbook.com/book/krzychb/esp8266wifi-library/details

5. Niveau Application



Browser
WEB

HTML

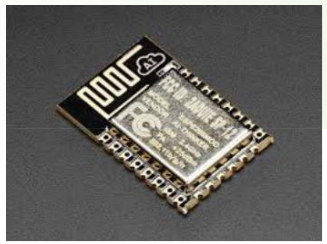


-On met le module ESP en mode « SoftAP » ou « Serveur * »
-On prépare des fonctions qui seront envoyées au browser appelant
-On attend un appel
-Dès l'appel entrant on envoie la page
-En boucle, on interagit avec l'appelant
-Les périphériques sont activés selon besoins

Applic UDP
Ex. OSC



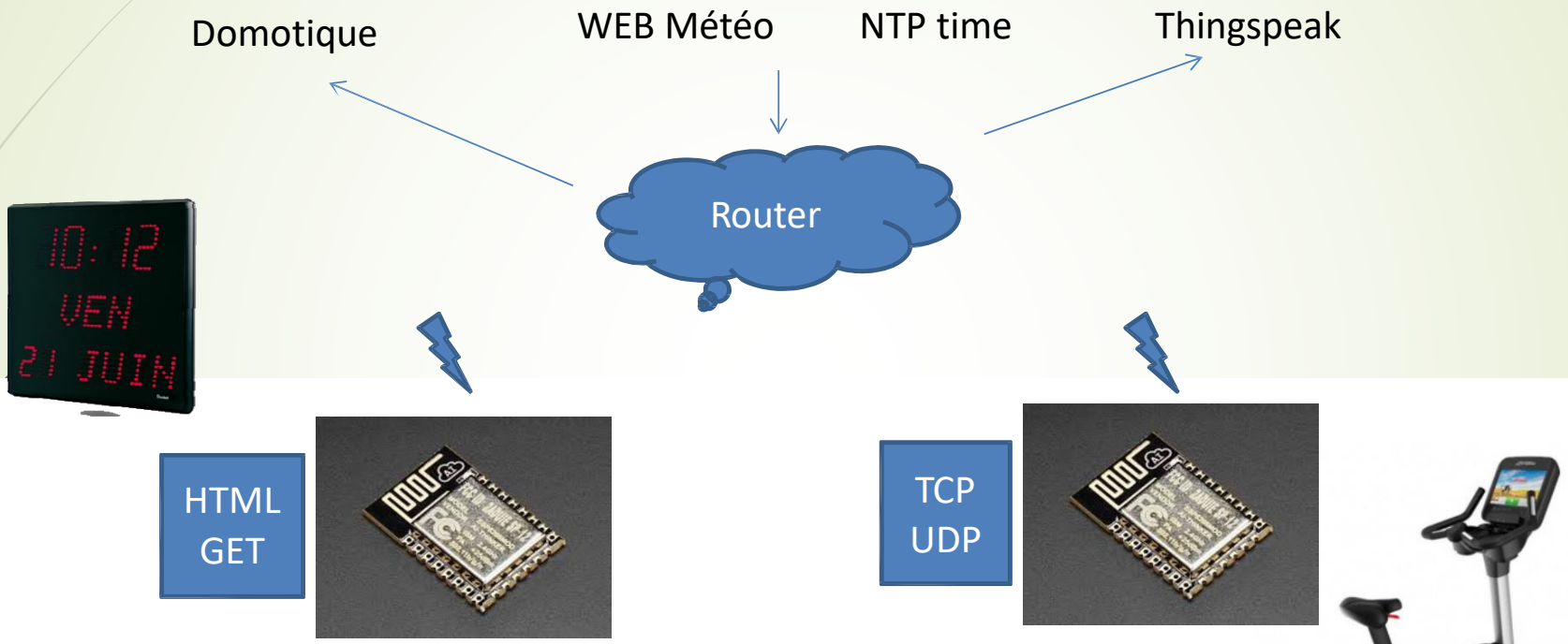
TCP
UDP



On ouvre une instance UDP
On se met en mode «SoftAP » ou « Server *»
On attend une connexion
-On lit instructions ou données entrantes
-On réagit sur SPI/I2C/UART etc
-On envoie les données vers l'application

* Mode server par un AP(Routeur)

5.1 Niveau Application



-On met ESP en mode « Client » par une AP(routeur ou PC)
-On se connecte sur le réseau local
-On envoie des données à un serveur (local ou à distance)
-On envoie une demande à un serveur (avec ou sans identification)
-On reçoit une réponse (XML,…)

6. Librairie exemple de code

```
#include <ESP8266WiFi.h>

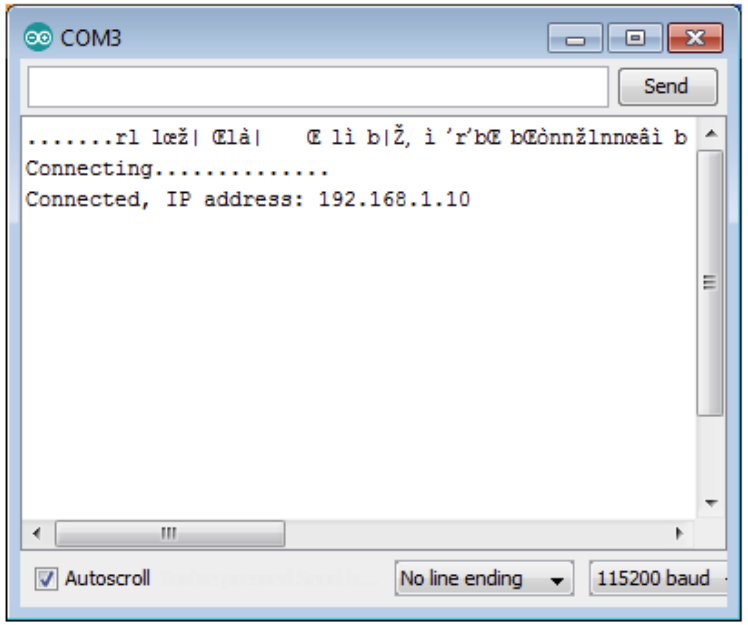
void setup()
{
  Serial.begin(115200);
  Serial.println();

  WiFi.begin("network-name", "pass-to-network");

  Serial.print("Connecting");
  while (WiFi.status() != WL_CONNECTED)
  {
    delay(500);
    Serial.print(".");
  }
  Serial.println();

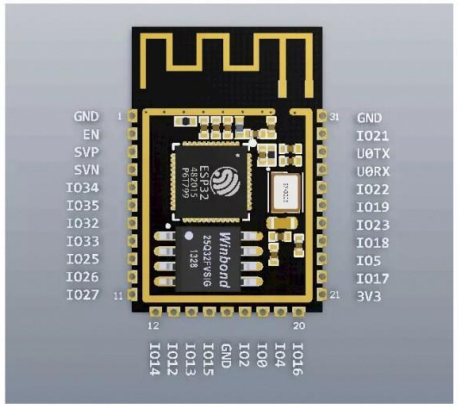
  Serial.print("Connected, IP address: ");
  Serial.println(WiFi.localIP());
}

void loop() {}
```



On voit ici la connexion sur un AP
-wifi.begin(….) se connecte sur un AP connu (SSID et PWD)
-puis attend que la connexion soit établie avant de continuer
! il manque le code à exécuter après.

7. Future produits ESP3212



- 12-bit SAR ADC 18 canaux
- 2 × convertisseur 8-bit D/A
- 10 × capteurs « touch »
- Capteur de température
- 4 × SPI, 2 × I2S, 2 × I2C, 3 × UART
- 1 host (SD/eMMC/SDIO), 1 slave (SDIO/SPI)
- Interface Ethernet MAC avec DMA dédié et support IEEE 1588
- CAN 2.0
- Interface InfraRouge (TX/RX)
- PWM Moteur, LED PWM up to 16 channels
- Capteur de Hall
- Ultra low power analog pre-amplifier

8. Questions/Réponses

Annexe A: Liens

https://nurdspace.nl/ESP8266#Technical_Overview

http://benlo.com/esp8266/esp8266QuickStart.html

https://github.com/esp8266

https://github.com/esp8266/Arduino

français

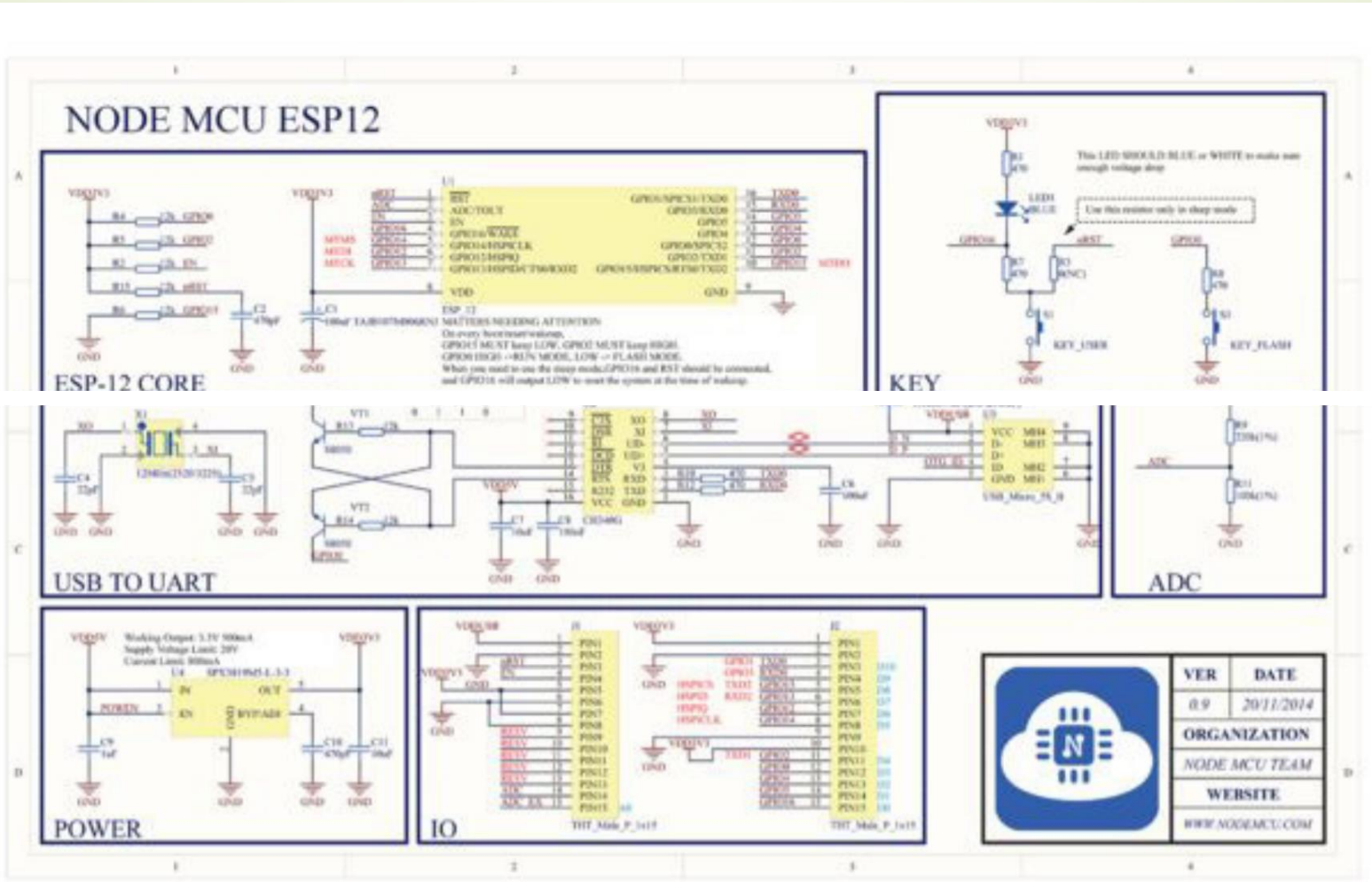
http://easydomoticz.com/forum/viewtopic.php?t=1840

https://www.gitbook.com/book/krzychb/esp8266wifi-library/details

Annexe B: Consommation

Mode	Min	Typical	Max	Units
802.11b, CCK 1Mbps, POUT=+19.5dBm		215		mA
802.11b, CCK 11Mbps, POUT=+18.5dBm		197		mA
802.11g, OFDM 54Mbps, POUT=+16dBm		145		mA
802.11n, MCS7, POUT =+14dBm		135		mA
802.11b, packet size of 1024 bytes, -80dBm		60		mA
802.11b, packet size of 1024 bytes, -70dBm		60		mA
802.11b, packet size of 1024 bytes, -65dBm		62		mA
Standby		0.9		uA
Deep sleep		10		mA
Saving mode DTIM 1		1.2		mA
Saving mode DTIM 3		0.86		mA
Shutdown		0.5		uA

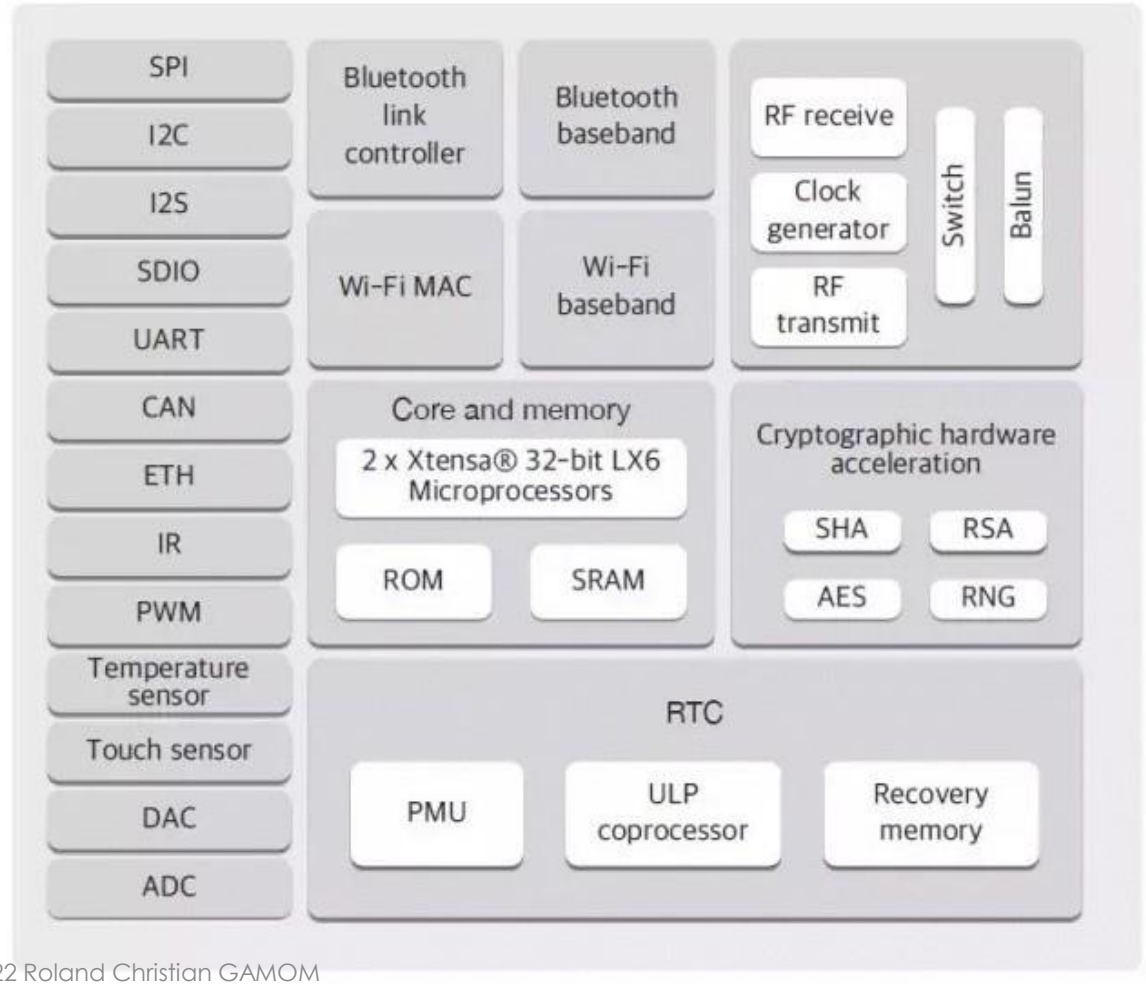
Annexe C: Node MCU Schemat c i
ES8266 Node MCU



Annexe D: AT commands

Commands	Description	Type	Set/Execute	Inquiry	test	Parameters	Examples
AT+RST	restart the module	basic	-	-	-	-	
AT+CWMODE	wifi mode	wifi	AT+CWMODE=<mode>	AT+CWMODE?	AT+CWMODE=?	1= Sta, 2= AP, 3=both	
AT+CWJAP	join the AP	wifi	AT+ CWJAP =<ssid>,<pwd >	AT+ CWJAP?	-	ssid = ssid, pwd = wifi password	
AT+CWLAP	list the AP	wifi	AT+CWLAP				
AT+CWQAP	quit the AP	wifi	AT+CWQAP	-	AT+CWQAP=?		
AT+ CWSAP	set the parameters of AP	wifi	AT+ CWSAP= <ssid>, <pwd>,<chl>, <ecn>	AT+ CWSAP?		ssid, pwd, chl = channel, ecn = encryption	Connect to your router: ; AT+CWJAP="YOURSSID","helloworld"; and check if connected: AT+CWJAP?
AT+ CIPSTATUS	get the connection status	TCP/IP	AT+ CIPSTATUS				
AT+CIPSTART	set up TCP or UDP connection	TCP/IP	1)single connection (+CIPMUX=0) AT+CIPSTART= <type>, <addr>,<port>; 2) multiple connection (+CIPMUX=1) AT+CIPSTART= <id> <type>,<addr>, <port>	-	AT+CIPSTART=?	id = 0-4, type = TCP/UDP, addr = IP address, port= port	Connect to another TCP server, set multiple connection first: AT+CIPMUX=1; connect: AT+CIPSTART=4,"TCP","X1.X2.X3.X4",9999
AT+CIPSEND	send data	TCP/IP	1)single connection(+CIPMUX=0) AT+CIPSEND= <length>; 2) multiple connection (+CIPMUX=1) AT+CIPSEND= <id>, <length>		AT+CIPSEND=?		send data: AT+CIPSEND=4,15 and then enter the data
AT+CIPCLOSE	close TCP or UDP connection	TCP/IP	AT+CIPCLOSE=<id> or AT+CIPCLOSE		AT+CIPCLOSE=?		
AT+CIFSR	Get IP address	TCP/IP	AT+CIFSR		AT+ CIFSR=?		
AT+ CIPMUX	set mutiple connection	TCP/IP	AT+ CIPMUX=<mode>	AT+ CIPMUX?		0 for single connection 1 for mutiple connection	
AT+ CIPSERVER	set as server	TCP/IP	AT+ CIPSERVER= <mode>[,<port>]			mode 0 to close server mode, mode 1 to open; port = port	turn on as a TCP server: AT+CIPSERVER=1,8888, check the self server IP address: AT+CIFSR=?

Annexe E: ESP3212 block diagram



Contenu ESP 32

- Présentation de l'ESP 32
- Différentes cartes ESP et choix
- Les librairies ARDUINO pour l'ESP
- Le câblage de l'ESP
- Premier Programme ESP

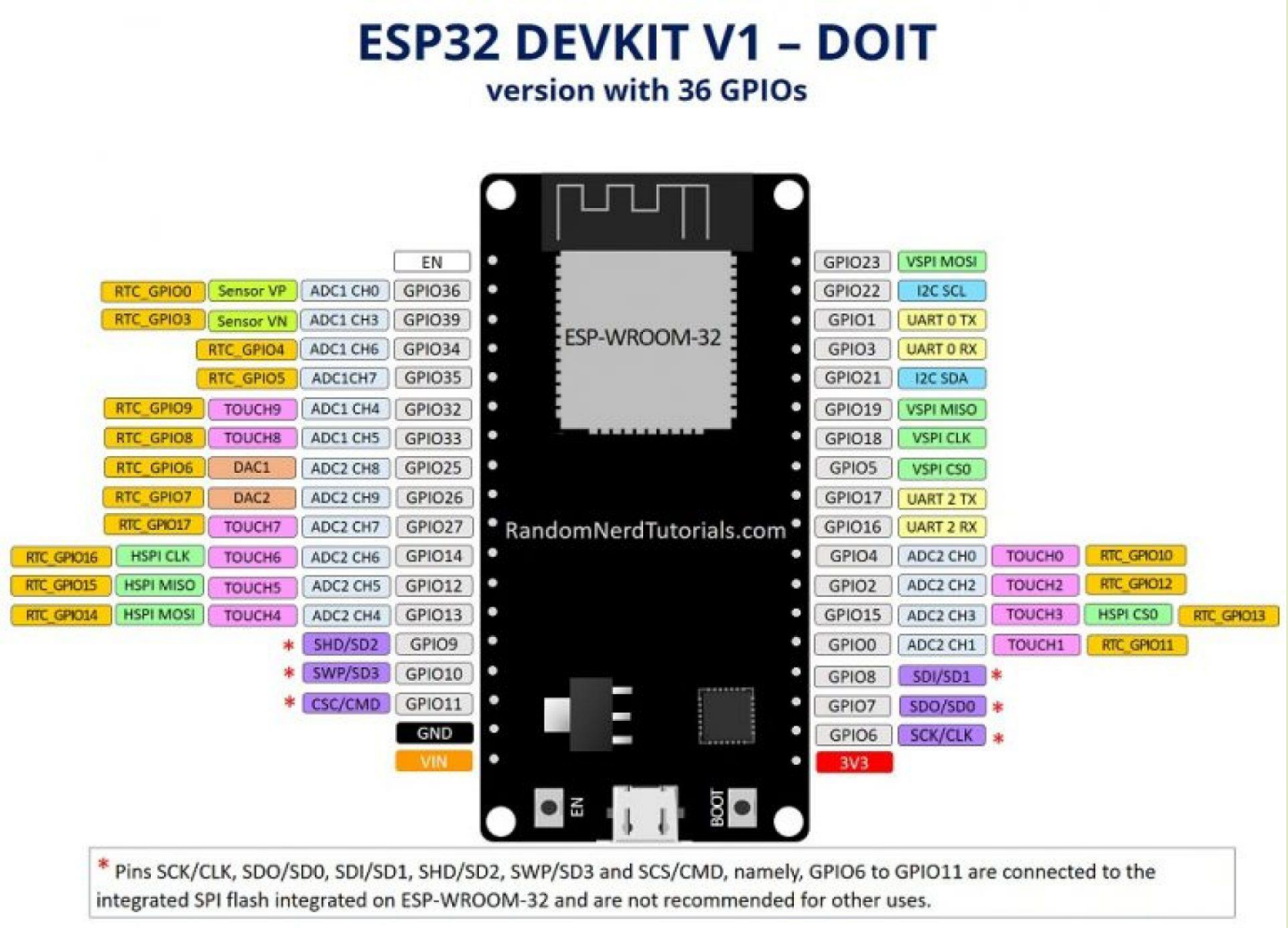
Présentation de l'ESP 32

- L'ESP32 est un SOC développé par la société Espressif dédié à l'internet des objets (IoT) et plus particulierement les communications sans fil Wifi et Bluetooth pour un coût réduit. Il a été décliné en version module l'ESP-WROOM-32 qui a lui même été intégré par différents fabricants (Essentiellement Chinois) sur des cartes de développement



Quelques cartes ESP32

- La DOIT Esp32 DevKit v1 est l'une des cartes de développement créées par DOIT pour évaluer le module ESP-WROOM-32..



Quelques cartes ESP32: ESP32 DEVKIT

- **Remarques :**
- L'entrée **Vin** du «ESP32 DEVKIT DOIT» fonctionne entre 4,5 et 12 volts. Un régulateur (NCP1117) sur le circuit abaisse la tension à 3,3 volts et fournit un courant maximum de 800ma.
- Le ESP32 fonctionne entre 2,2 et 3,6 volts.
- **Les broches d'entrées/sortie du «ESP32 DEVKIT DOIT» ne supportent pas le 5 volts.** Le voltage à l'entrée ne doit pas dépasser 3,6 volts.
- Les GPIO 36, 39, 34 et 35 sont des broches d'entrée de signal seulement (I). Elles ne possèdent pas de résistances internes de pull-up et pull-down.
- Les broches GPIO < 34 peuvent fonctionner en PWM
- **Le courant maximal absolu par GPIO est de 40 mA**

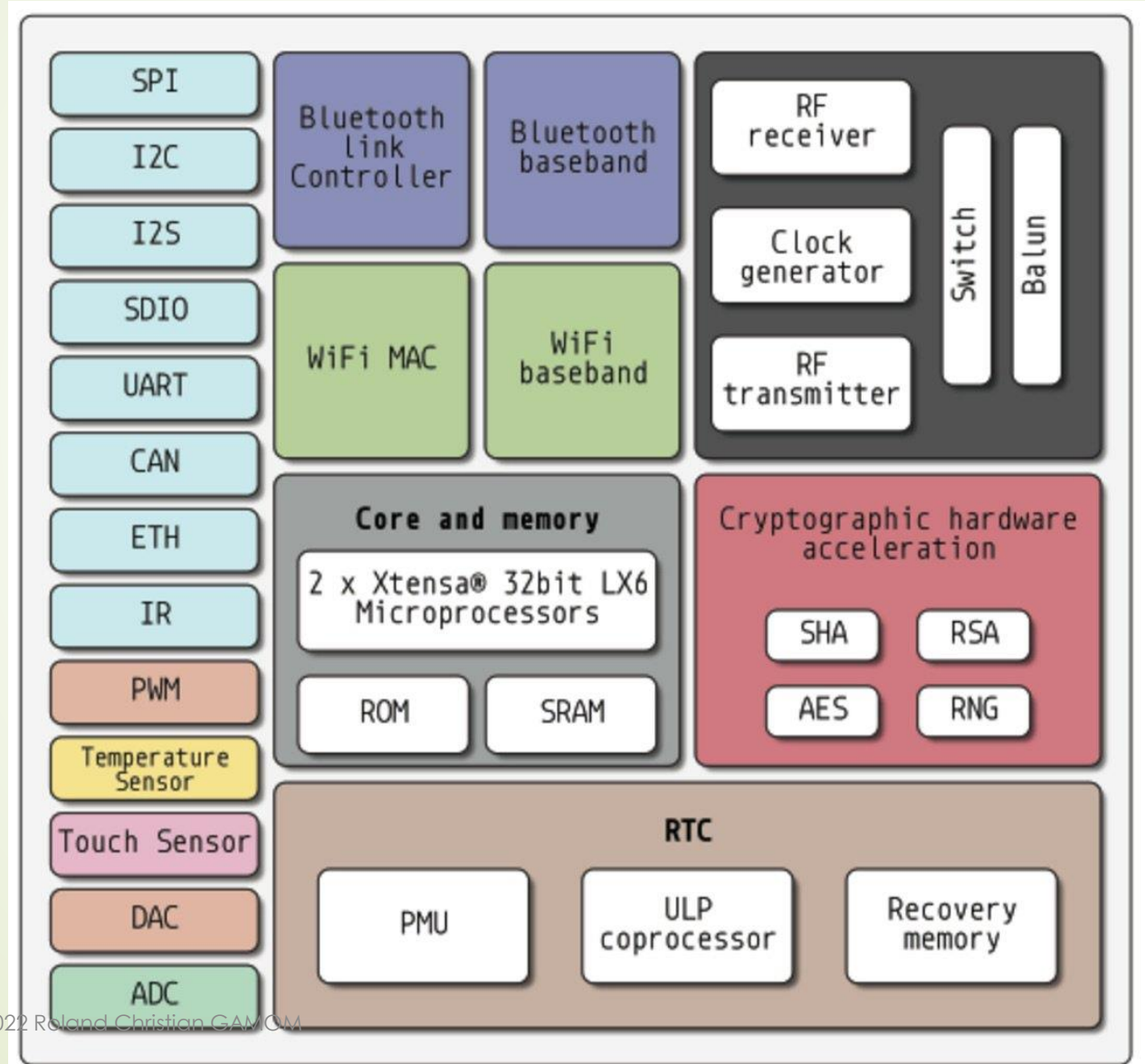
Caractéristiques de l'ESP 32DEVKITV1

- **Connectivité sans fil**
  - **WiFi**: débit de données de 150,0 Mbps avec HT40
  - **Bluetooth**: BLE (Bluetooth Low Energy) et Bluetooth classique
- **Processeur**: microprocesseur LX6 32 bits Dual-Core Tensilica Xtensa, fonctionnant à 160 ou 240 MHz
- **ROM**: 448 Ko
- **SRAM**: 520 Ko
- **Basse consommation**: garantie que vous pouvez toujours utiliser les conversions ADC, par exemple pendant le sommeil profond.
- **Entrée / sortie périphérique**:
  - Minuteries et chien de garde
  - Horloge temps réel
  - Convertisseur analogique-numérique (ADC) 12 bits
  - Convertisseur numérique-analogique (DAC)
  - Capteurs intégrés (Température, effet Hall)
  - Détecteur tactile capacitif

Caractéristiques de l'ESP 32DEVKITV1

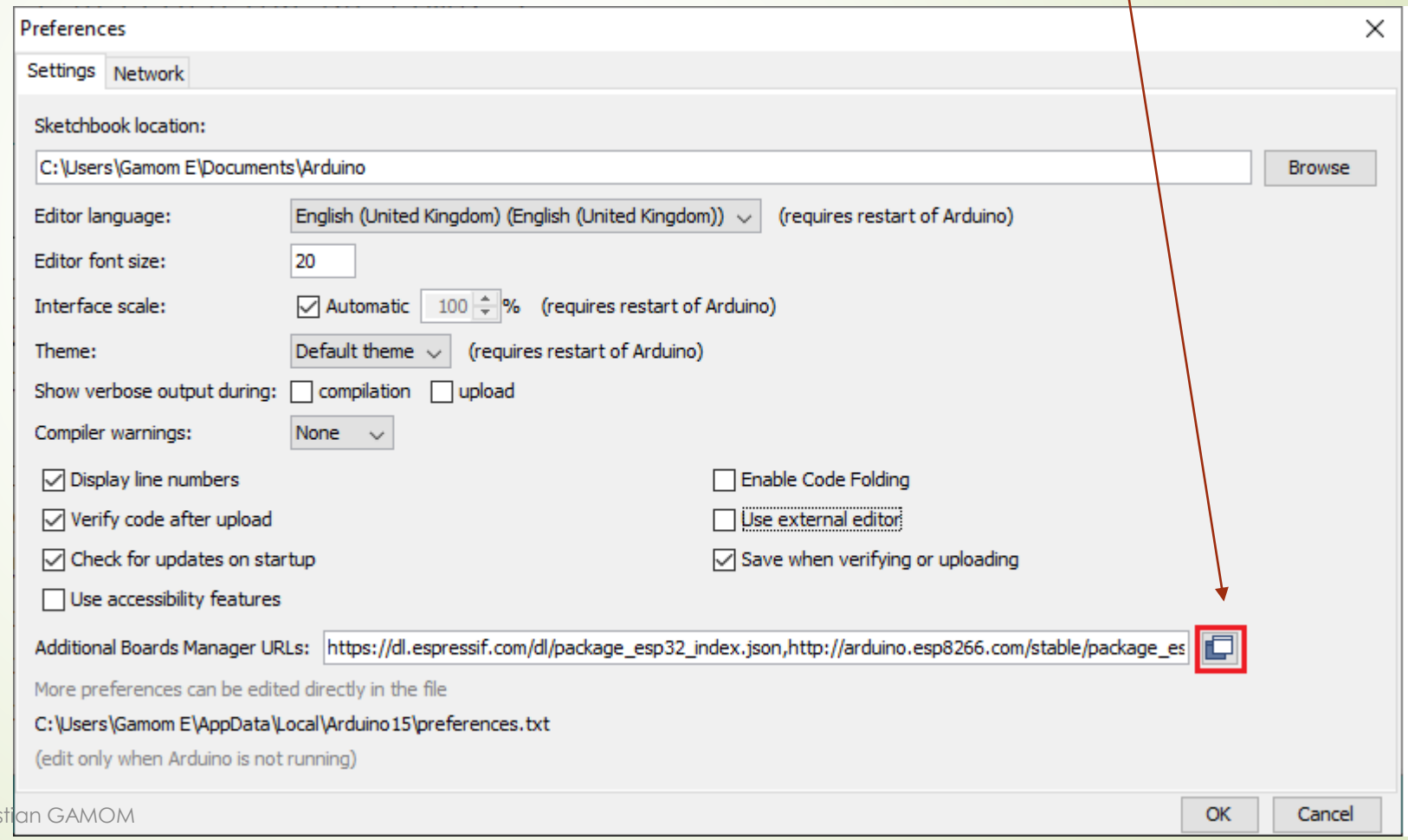
- Co-processeur Ultra Low Power (ULP)
- Interface Ethernet MAC
- Contrôleur hôte SD / SDIO / MMC
- Émetteur récepteur universel asynchrone (UART)
- Interface 2 fils (I2C)
- Interface son interconnecté intégré (I2S)
- Interface de périphérique série (SPI)
- Télécommande infrarouge
- Compteur d'impulsions
- Modulation de largeur d'impulsion (PWM) 16 sorties
- Accélérateur Matériel
- **Sécurité**:
  - accélérateurs matériels pour AES et SSL / TLS

ESP32DEVKITV1 Composantes Internes



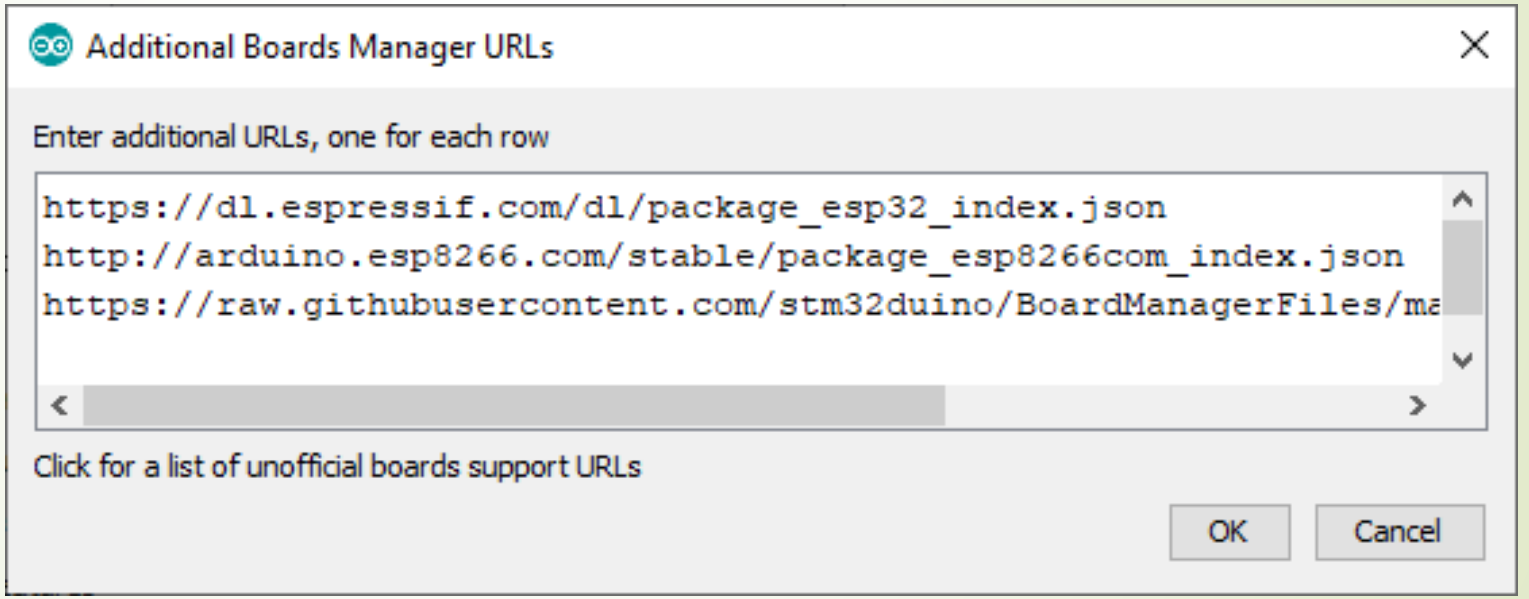
Installation ESP32 pour IDE Arduino

- 1Ouvrez le panneau des préférences depuis le menu Arduino
- cliquez sur le bouton situé à droite de l'option URL de gestionnaire de cartes supplémentaires



Installation ESP32 pour IDE Arduino

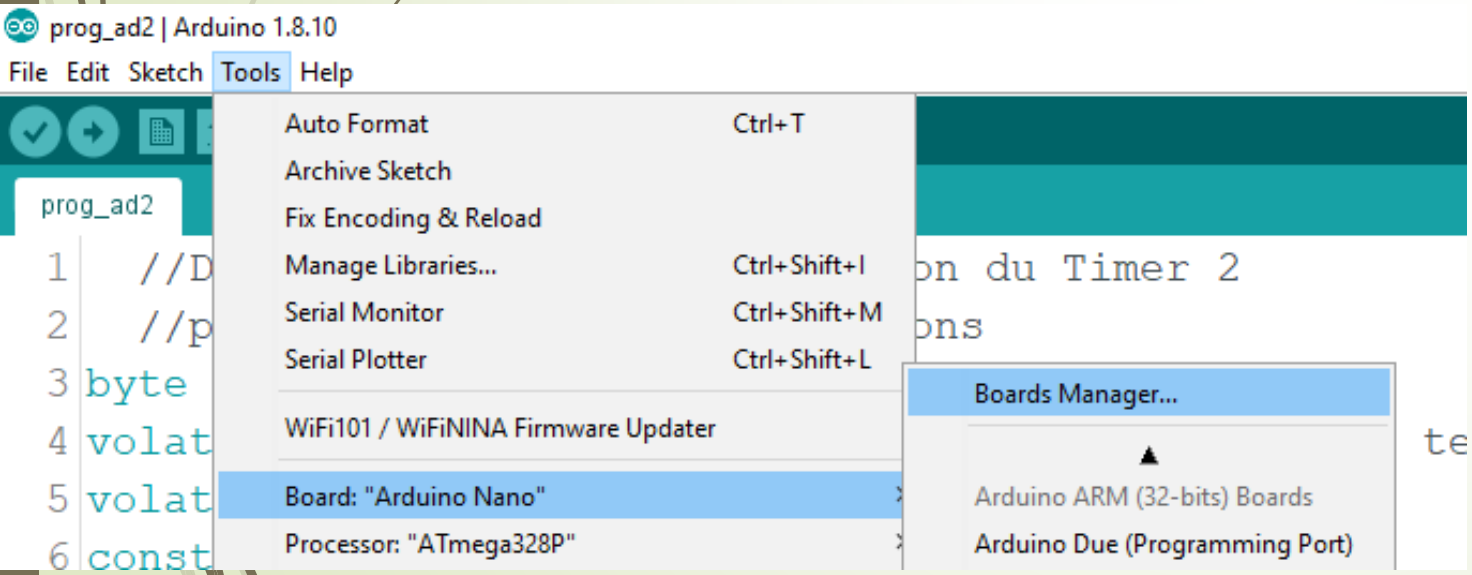
- Dans la boîte de dialogue qui s'ouvre, ajoutez le lien ci-dessous puis cliquez sur OK
  - https://dl.espressif.com/dl/package_esp32_index.json
- Attention taper, une ligne par URL si vous en avez plusieurs

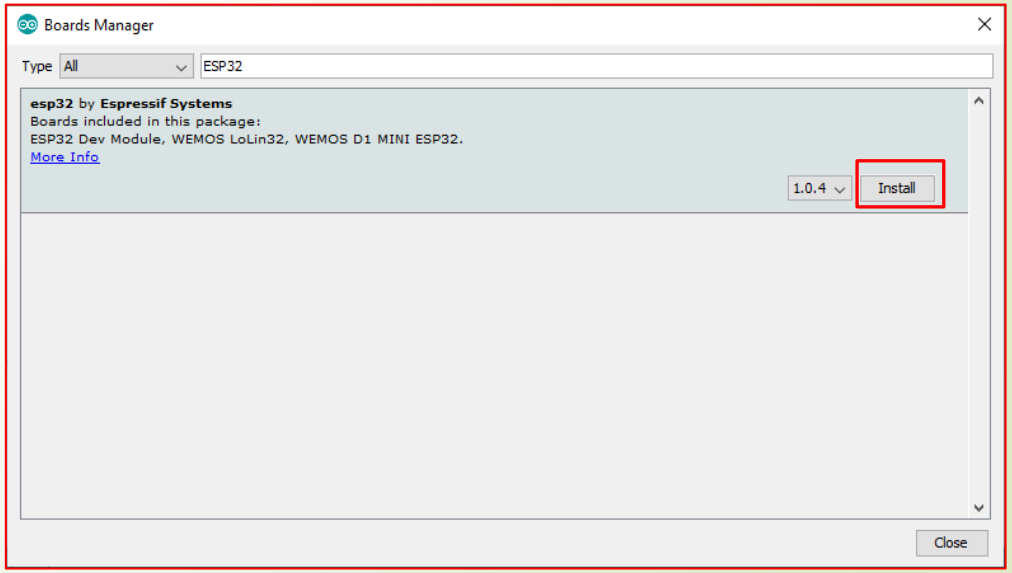


- Le lien vers le dépôt est ajouté à la liste. Fermer les paramètres en cliquant sur OK

Installation ESP32 pour IDE Arduino

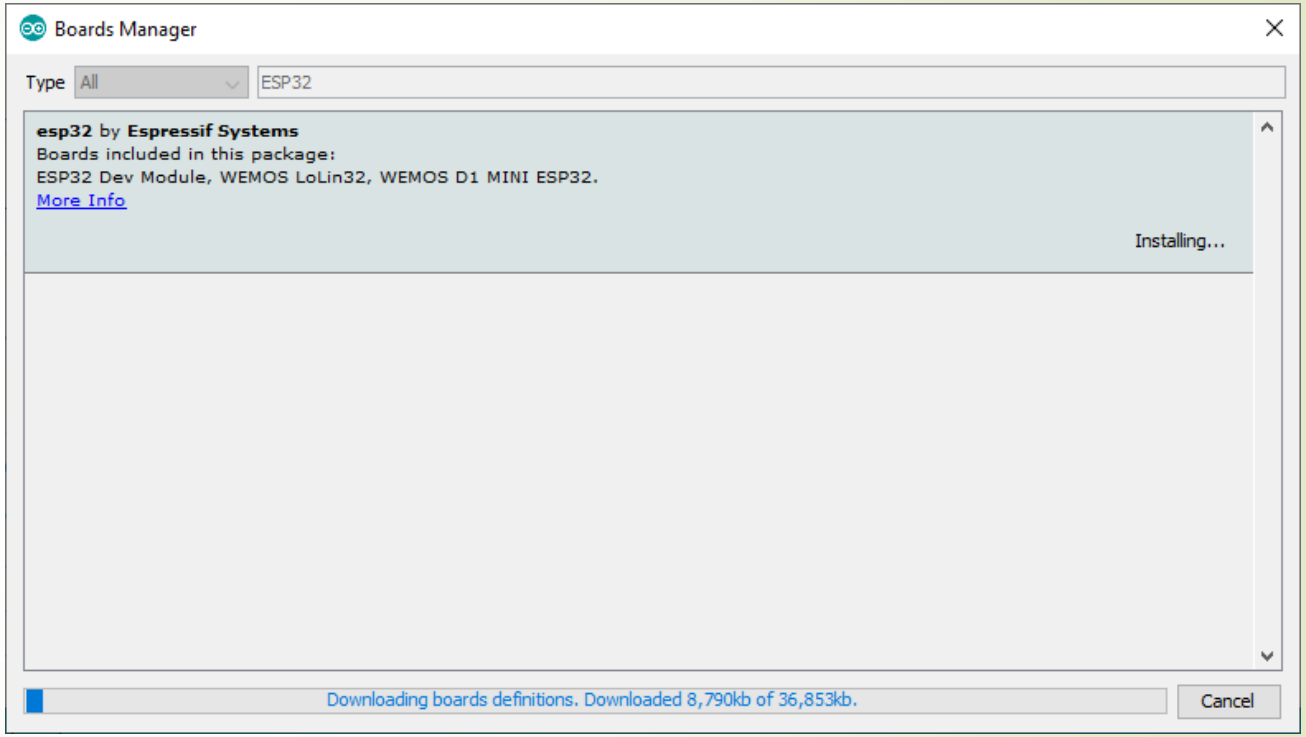
- Ouvrez le gestionnaire de carte depuis le menu Outils(Tools) puis Type de carte (Board…)
- Cherchez le SDK avec le mot clé ESP32. Choisissez le package de cartes nommé ESP32 by Espressif Systems puis cliquez sur installer.
- Maintenant que les cartes ESP32 sont gérées depuis le gestionnaire de carte, les librairies pourront être mises à jour dès qu'une nouvelle version est disponible.





Installation ESP32 pour IDE Arduino

- Cherchez le SDK avec le mot clé ESP32. Choisissez le package de cartes nommé ESP32 by Espressif Systems puis cliquez sur installer.
- Maintenant que les cartes ESP32 sont gérées depuis le gestionnaire de carte, les librairies pourront être mises à jour dès qu'une nouvelle version est disponible.
- Attendez que le framework soit téléchargé et installé depuis Internet. Le processus d'installation est totalement automatisé. Il n'y a absolument rien à faire de plus.



Installation ESP32 pour IDE Arduino

- Une fois l'installation terminée, il n'est pas nécessaire de redémarrer l'IDE Arduino.
- Ouvrez le menu Outils puis le sous-menu Type de carte. Faites défiler la liste, si l'installation s'est correctement déroulée, vous devriez pouvoir sélectionner votre carte de développement ESP32 dans la liste.
- Si vous ne trouvez pas votre carte dans la liste, vous pouvez essayer une configuration générique, par exemple :
- ESP32 Dev Module
- ESP32 Wrover Module
- NodeMCU-32S

Différentes cartes ESP32

Le framework prend en charge les cartes de développement construites autour des puces suivantes.

MODULE	PUCES	MÉMOIRE FLASH (MB)	PSRAM (MB)	ANTENNE
ESP32-WROOM-32	ESP32-D0WDQ6	4	–	PCB
ESP32-WROOM-32D	ESP32-D0WD	4, 8 ou 16	–	PCB
ESP32-WROOM-32U	ESP32-D0WD	4, 8 ou 16	–	U.FL
ESP32-SOLO-1	ESP32-S0WD	4	–	PCB
ESP32-WROVER (PCB)	ESP32-D0WDQ6	4	8	PCB
ESP32-WROVER (IPEX)	ESP32-D0WDQ6	4	8	U.FL
ESP32-WROVER-B	ESP32-D0WD	4, 8 ou 16	8	PCB
ESP32-WROVER-IB	ESP32-D0WD	4, 8 ou 16	8	U.FL

Programme exemple

- **Vérifier que tout fonctionne en faisant clignoter une Led**
- L'ESP32 se programme exactement comme un ESP8266. Nous allons ici simplement vérifier que tout est correctement installé. Créez un nouveau projet et collez le code suivant qui fait clignoter une Led branchée entre la broche 15 et le GND.
- ESP32 DevKit V1 est livré avec une LED bleue intégrée qui est connectée à GPIO2 comme dans l'exemple de clignotement ESP8266.

```
/* * ESP32 Dev Board Test
* Led blinking  */
int ledPin = 2;
void setup() { pinMode(ledPin, OUTPUT); Serial.begin(115200); }
void loop() {
Serial.println("Hello ESP32!");
digitalWrite(ledPin, HIGH); delay(500); digitalWrite(ledPin,
LOW); delay(500); }
```

Téléversement d'un programme dans l'ESP32

- Sélectionner le port COMx dans le menu outils (tools)
- Cliquer sur upload sketch
- Appuyer sur le bouton « boot » de la carte ESP32
- En cas de difficulté à transférer le programme, connecter un condensateur de 10µF entre la broche EN et la broche GND

Configurer l'entrée numérique de l'ESP32

- Avant de pouvoir utiliser sur une entrée numérique du GPIO, il faut tout d'abord configurer la broche comme une entrée à l'aide de la fonction pinMode() comme ceci
  - pinMode(broche, mode);
- La méthode pinMode a deux arguments
  - Le numéro de la broche
  - Le mode
- 3 modes sont disponibles pour configurer la broche comme une entrée numérique
  - INPUT En fonction du périphérique d'entré utilisé, il faudra adapter le circuit..
  - INPUT_PULLUP On utilise la résistance de tirage (PULL UP) de l'ESP32
  - INPUT_PULLDOWN On utilise la résistance de rappel (PULL DOWN) de l'ESP32

Appel d'une Interruption avec ESP32

- Comment ajouter une interruption externe à un projet ESP32 ?
- Ensuite on assigner une fonction qui sera exécutée dès qu'un événement est détecté sur la broche à l'aide de la méthode attachInterrupt(GPIO, FUNCTION, MODE). La méthode nécessite 3 arguments :
  - **GPIO** La broche qui déclenche l'événement. C'est la broche que l'on a configuré précédemment
  - **FUNCTION** la fonction à exécuter lorsqu'un événement survient
  - **MODE** mode de déclenchement
- 5 modes de déclenchement sont possibles

- **LOW** pour déclencher l'interruption chaque fois que la broche est à l'état bas (LOW)
- **HIGH** pour déclencher l'interruption chaque fois que la broche est à l'état haut (HIGH)
- **CHANGE** pour déclencher l'interruption chaque fois que la broche change d'état. Par exemple lorsqu'elle passer de HIGH à LOW ou LOW à HIGH
- **FALLING** pour quand la broche passe de HIGH à LOW. C'est la détection du front descendant.
- **RISING** pour déclencher lorsque la broche passe de LOW à HIGH. C'est la détection du front montant.

Interruption Timer exemple ESP32

- Code

```
volatile int interruptCounter;
int totalInterruptCounter;

hw_timer_t * timer = NULL;
portMUX_TYPE timerMux = portMUX_INITIALIZER_UNLOCKED;

void IRAM_ATTR onTimer() {
 portENTER_CRITICAL_ISR(&timerMux);
 interruptCounter++;
 portEXIT_CRITICAL_ISR(&timerMux);
}

void setup() {

 Serial.begin(115200);

 timer = timerBegin(0, 80, true);
 timerAttachInterrupt(timer, &onTimer, true);
 timerAlarmWrite(timer, 1000000, true);
 timerAlarmEnable(timer);
}
```

Interruption Timer exemple ESP32

```
void loop() {

  if (interruptCounter > 0) {

    portENTER_CRITICAL(&timerMux);
    interruptCounter--;
    portEXIT_CRITICAL(&timerMux);

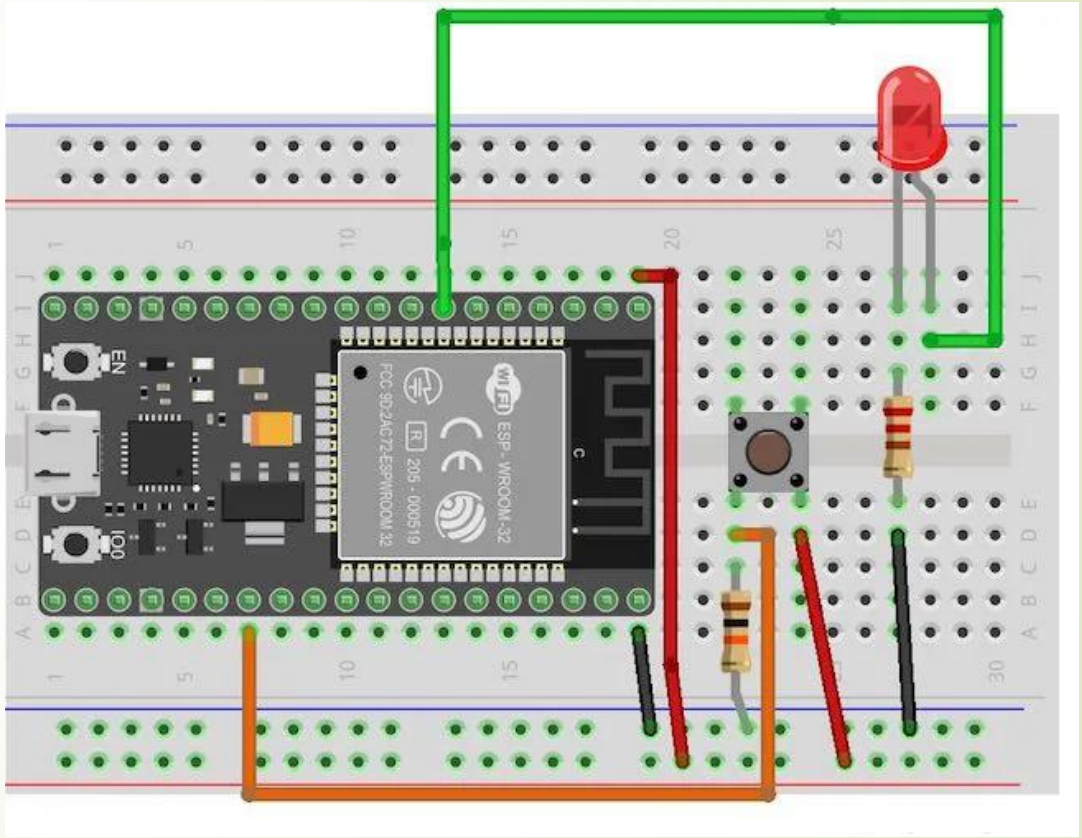
    totalInterruptCounter++;

    Serial.print("An interrupt as occurred. Total number: ");
    Serial.println(totalInterruptCounter);

  }
}
```

Montage pour utiliser l'ESP32

- Circuit
- Le bouton poussoir est connecté à l'entrée numérique 4. La LED est connectée à la sortie 32.
- Vous pouvez tester le fonctionnement du programme en l'absence de résistance de tirage (PULL_UP). Vous pouvez tester en utilisant la résistance de tirage (PULL_UP) interne de l'ESP32 ou ajouter une résistance au circuit (10 kΩ par exemple).



Types de données ESP32

- Sd

Data Type	Uno, Mega 2560	ESP8266	ESP32
bool	1		
boolean			
byte			
char			
unsigned char			
uint8_t			
short	2		2
uint16_t			
word			4
int	2	4	
unsigned int			
size_t			
float	4		
long			
unsigned long			
uint32_t			
double	4	8	
long long	8		
unsigned long long			
uint64_t			

Commander une LED depuis le moniteur serie

- Créer un nouveau croquis sur l'IDE Arduino et collez le code suivant.
- Les commandes suivantes sont acceptées
  - led=on pour allumer la LED
  - led=off pour éteindre la LED
  - ledstatus pour connaître l'état de la LED (ON ou OFF) qui est stocké dans la variable led_status
- Tout autre commande retourne l'erreur Invalid command
- Avant de téléverser le code, modifiez la constante LED_PIN qui indique la broche sur laquelle la LED est branchée

Commander une LED depuis le moniteur serie


```
#include <Arduino.h>
#define LED_PIN 2
bool led_status = false;
String command;
void setup() { Serial.begin(115200);
pinMode(LED_PIN, OUTPUT); }
void send_led_status(){
 if ( led_status ) {
Serial.println("LED is ON"); }
else { Serial.println("LED is OFF"); }
}
void loop() {
 if(Serial.available())
        { command = Serial.readStringUntil('\n');
Serial.printf("Command received %s \n", command);
if(command.equals("led=on")){ digitalWrite(LED_PIN, HIGH); led_status = 1; }
else if(command.equals("led=off"))
{ digitalWrite(LED_PIN, LOW); led_status = 0; }
else if(command.equals("ledstatus"))
{ send_led_status(); }
 else{ Serial.println("Invalid command");
} }
}
```

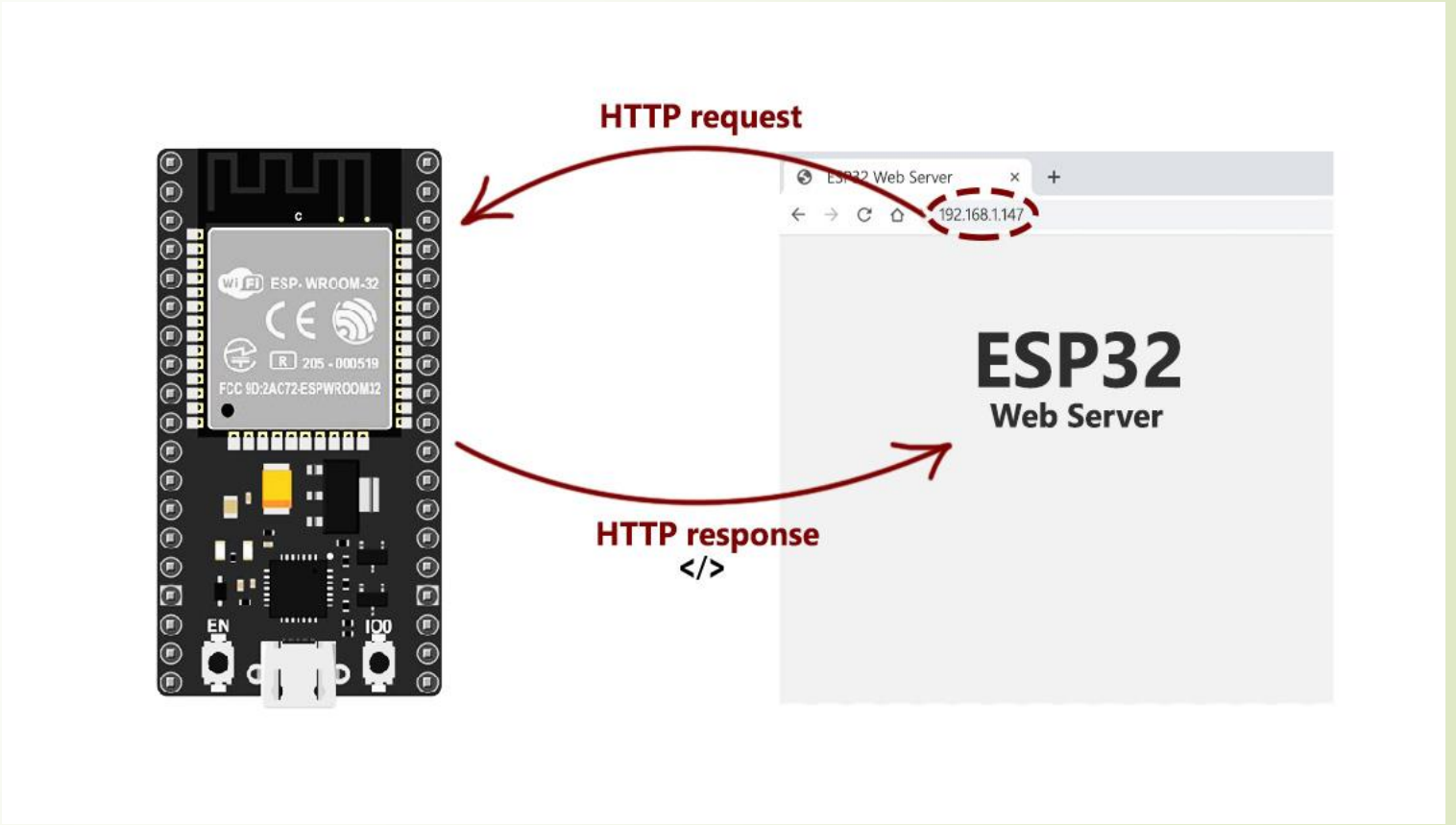
Serveur Web avec ESP32

- Qu'est-ce qu'un serveur Web?
- Le serveur Web est un programme pour envoyer et recevoir des informations en utilisant le protocole HTTP, il peut traiter les informations et les stocker. Le client d'un serveur web est le navigateur qui peut afficher les informations fournies par le serveur sur un écran de smartphone, d'une smart tv ou un d'un laptop.

- le serveur communique avec l'utilisateur via un protocole appelé Hypertext Transfer Protocol (HTTP).

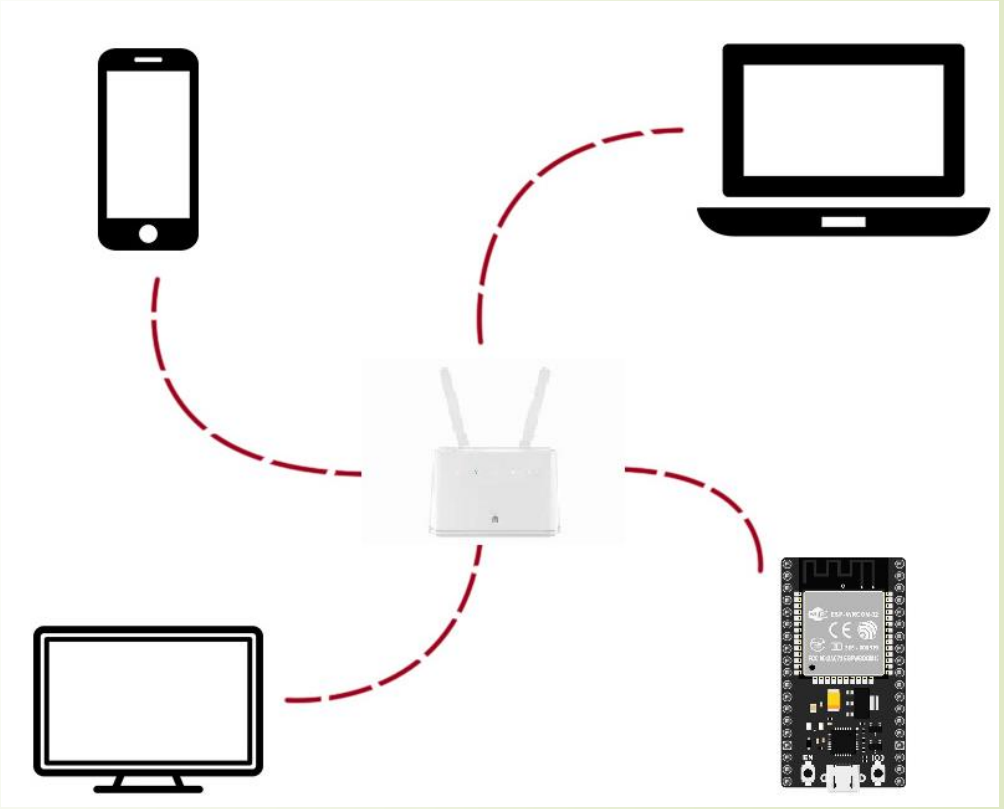
Fonctionnement du HTTP

- Lorsqu'une requête HTTP est envoyée à ce serveur (par exemple, son adresse est recherchée dans le navigateur), le serveur renvoie un code en réponse (par exemple, le code 200, qui signifie que la connexion est établie correctement, ou le code 404, qui indique que l'adresse n'est pas correcte).



Configuration de l'ESP32 en mode Station (STA)

- Dans ce cas, le module ESP32 est connecté au routeur Wi-Fi en tant que client et peut accéder à Internet via le routeur.



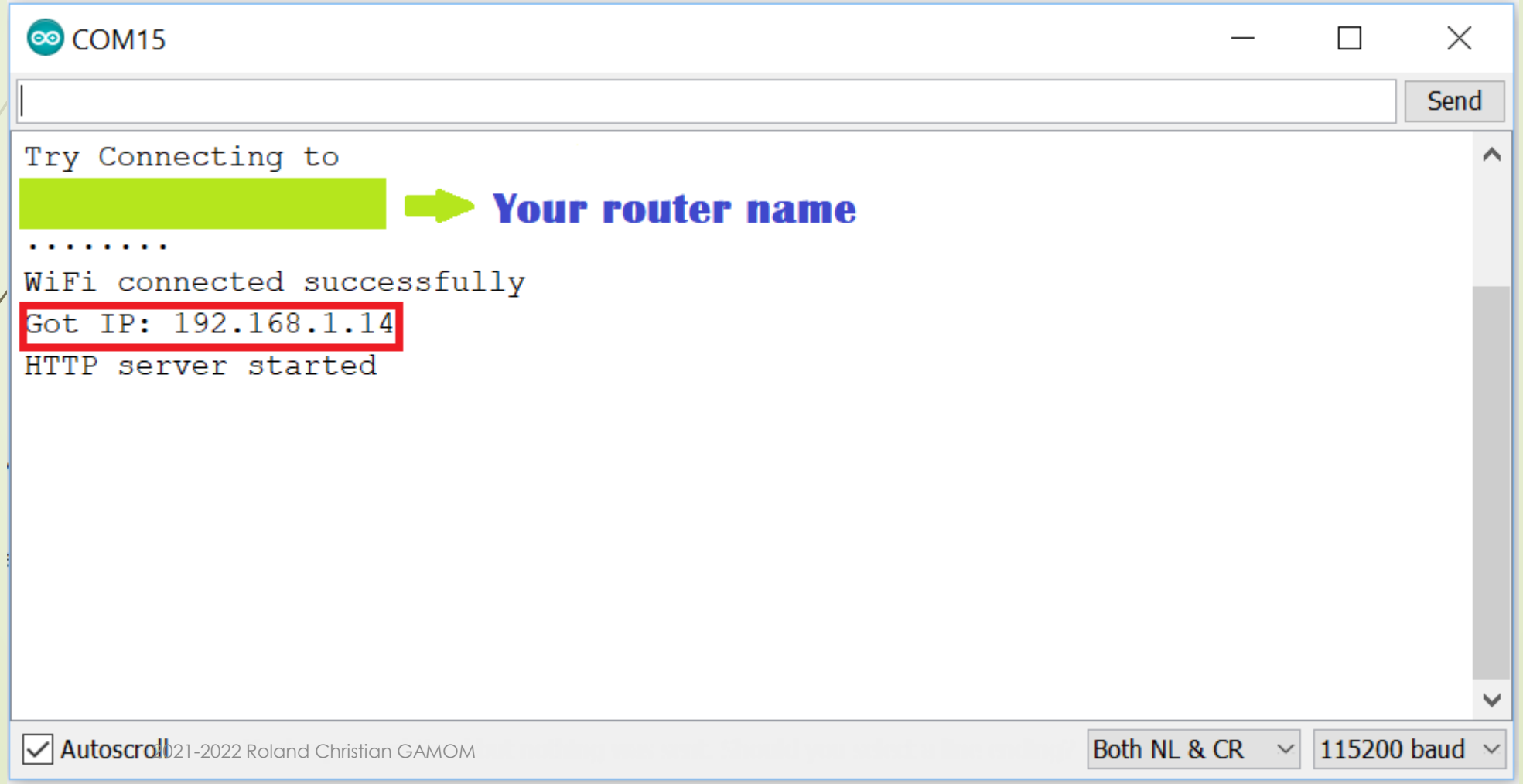
Configuration de l'ESP32 en mode Station (STA)

- Code Pour démarrer l'ESP32 en mode STA, téléchargez simplement le code suivant sur la carte.

```
#include <WiFi.h>
#include <WebServer.h>
// SSID & Password
 const char* ssid = "******";  // Enter your SSID here
const char* password = "******";  //Enter your Password here
WebServer server(80);  // Object of WebServer(HTTP port, 80 is default)
void setup() {  Serial.begin(115200);
Serial.println("Try Connecting to ");
Serial.println(ssid);  // Connect to your wi-fi modem
WiFi.begin(ssid, password);  // Check wi-fi is connected to wi-fi network
while (WiFi.status() != WL_CONNECTED) {  delay(1000);  Serial.print(".");  }
Serial.println("");  Serial.println("WiFi connected successfully");
Serial.print("Got IP: ");  Serial.println(WiFi.localIP());  //Show ESP32 IP on serial
server.on("/", handle_root);
 server.begin();  Serial.println("HTTP server started");  delay(100); }
 void loop() {  server.handleClient(); } // HTML & CSS contents which display on web server
String HTML = "<!DOCTYPE html>\ <html>\ <body>\
 <h1>Mon serveur Web avec ESP32 - Mode Station &#128522;</h1>\ </body>\ </html>";
// Handle root url (/)
void handle_root() {  server.send(200, "text/html", HTML); }
```

Configuration de l'ESP32 en mode Station (STA)

- Une fois que vous avez téléchargé le code, ouvrez la fenêtre du moniteur série. Si vous avez entré le SSID et le mot de passe corrects, après quelques secondes, ESP32 se connectera au routeur et vous donnera une adresse IP.



©2021-2022 Roland Christian GAMOM

Configuration de l'ESP32 en mode Station (STA)

- En entrant cette adresse IP dans votre navigateur, vous pouvez voir la page Web que vous venez de créer.

Stockage de donées non-volatiles avec ESP32

- ESP32 dispose de deux outils de stockage non volatiles populaires: les **preferences** et **SPIFFS**.
- Alors que les préférences sont généralement utilisées pour stocker des paires clé-valeur, SPIFFS (SPI Flash File System), comme son nom l'indique, est utilisé pour stocker des fichiers et des documents.
- Les préférences sont stockées dans une section de la mémoire flash principale avec le type comme données et le sous-type comme nvs. nvs signifie stockage non volatile.
- Par défaut, 20 Ko d'espace sont réservés aux préférences, n'essayez donc pas de stocker beaucoup de données volumineuses dans les préférences. Utilisez SPIFFS pour les données volumineuses (SPIFFS a 1,5 Mo d'espace réservé par défaut)
- Commencons par utiliser la classe Preferences, concentrons-nous sur les préférences.

Exemple d'utilisation de Preferences

- Cet exemple illustre l'utilisation de la classe Preferences pour enregistrer le nombre de fois que le module ESP32 a redémarrer

```
/*
 ESP32 startup counter example with Preferences library.
 created for arduino-esp32 09 Feb 2017  by Martin Sloup (Arcao)
 adapted by GAMOM Roland Christian
*/

#include <Preferences.h>

Preferences preferences;

void setup() {
  Serial.begin(115200);
  Serial.println();
```

Exemple d'utilisation de Preferences

```
#include <Preferences.h>

Preferences preferences;

void setup() {
  Serial.begin(115200);
  Serial.println();

  // Ouvre Preferences avec l'espace de nommage mon-app
namespace.
  // Chaque module application, bibliothèque, etc
  // doit utiliser un espace de nom pour éviter la collision
des clés-valeurs.
```

Exemple d'utilisation de Preferences

```
// Le stockage est ouvert en mode lecture-ecriture
 // (c'est pourquoi le second parametre est false).
 // Note: l'espace de nom est limité à 15 chars.
 preferences.begin("mon-app", false);

 // vider l'espace de nom ouvert
 //preferences.clear();

 // ou vider la clé counter seulement
 //preferences.remove("counter");
```

Exemple d'utilisation de Preferences

```
  // lire la valeur de counter, si la clé est absente , renvoie
0 par défaut
  // Note: les nom de clé sont limités à 15 chars.
  unsigned int counter = preferences.getUInt("counter", 0);

  // Incremeter counter
  counter++;

  // Print the counter to Serial Monitor
  Serial.printf("valeur actuelle de counter : %u\n", counter);

  // Enregistrer counter dans les Preferences
  preferences.putUInt("counter", counter);
```

Exemple d'utilisation de Preferences

```
  // fermer les Preferences
  preferences.end();

  // Wait 10 seconds
  Serial.println("Redémarage dans 10 secondes...");
  delay(10000);

  // Restart ESP
  ESP.restart();
}

void loop() {}
```

Principales methodes de Preferences

```
size_t putChar(const char* key, int8_t value);
size_t putUChar(const char* key, uint8_t value);
size_t putShort(const char* key, int16_t value);
size_t putUShort(const char* key, uint16_t value);
size_t putInt(const char* key, int32_t value);
size_t putUInt(const char* key, uint32_t value);
size_t putLong(const char* key, int32_t value);
size_t putULong(const char* key, uint32_t value);
size_t putLong64(const char* key, int64_t value);
size_t putULong64(const char* key, uint64_t value);
size_t putFloat(const char* key, float_t value);
size_t putDouble(const char* key, double_t value);
size_t putBool(const char* key, bool value);
size_t putString(const char* key, const char* value);
size_t putString(const char* key, String value);
size_t putBytes(const char* key, const void* value, size_t len);

bool isKey(const char* key);
```

Principales methodes de Preferences

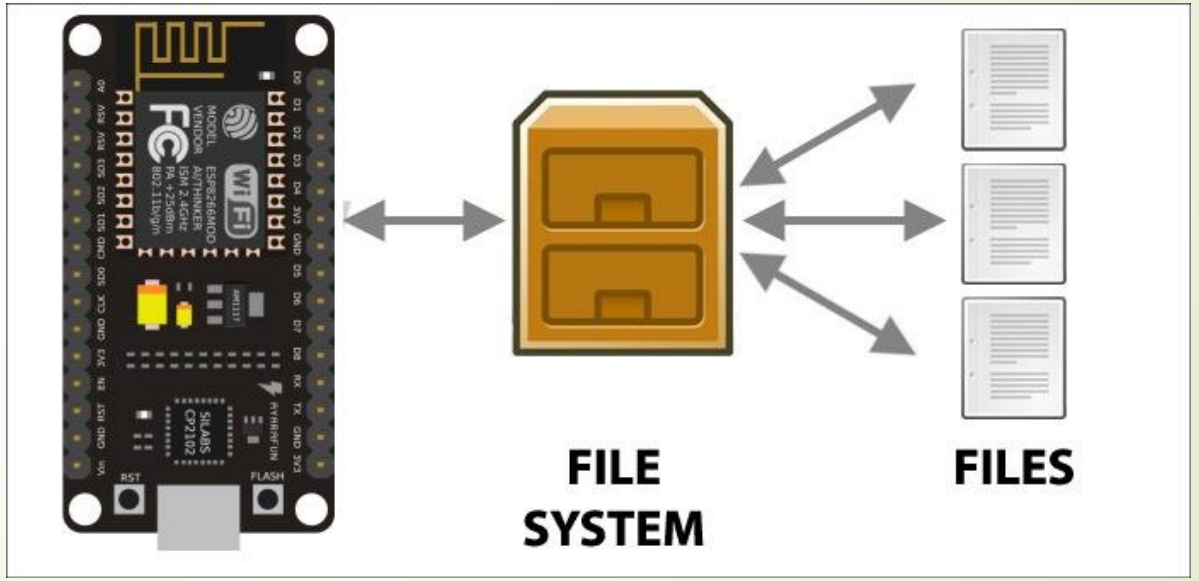
```
PreferenceType getType(const char* key);
int8_t getChar(const char* key, int8_t defaultValue = 0);
uint8_t getUChar(const char* key, uint8_t defaultValue = 0);
int16_t getShort(const char* key, int16_t defaultValue = 0);
uint16_t getUShort(const char* key, uint16_t defaultValue = 0);
int32_t getInt(const char* key, int32_t defaultValue = 0);
uint32_t getUInt(const char* key, uint32_t defaultValue = 0);
int32_t getLong(const char* key, int32_t defaultValue = 0);
uint32_t getULong(const char* key, uint32_t defaultValue = 0);
int64_t getLong64(const char* key, int64_t defaultValue = 0);
uint64_t getULong64(const char* key, uint64_t defaultValue = 0);
float_t getFloat(const char* key, float_t defaultValue = NAN);
double_t getDouble(const char* key, double_t defaultValue = NAN);
bool getBool(const char* key, bool defaultValue = false);
size_t getString(const char* key, char* value, size_t maxLen);
String getString(const char* key, String defaultValue = String());
size_t getBytesLength(const char* key);
size_t getBytes(const char* key, void * buf, size_t maxLen);
```

ESP32 et SPIFFS pour stocker les fichiers

- SPIFFS (Serial Peripheral Interface Flash File Storage), est utilisé pour stocker des données plus volumineuses sous forme de fichiers.
- SPIFFS est comparable à une très petite carte SD intégrée à la puce ESP32 elle-même.
- Par défaut, environ 1,5 Mo de mémoire flash intégrée est alloué à SPIFFS. Vous pouvez le voir par vous-même dans Arduino en ouvrant Outils -> « **Partition scheme** » Schéma de partition.
- Vous pouvez voir qu'il existe plusieurs autres options de partition disponibles. La modification du schéma de partition ne sera de toute façon pas nécessaire pour la plupart de vos applications.

ESP32 et SPIFFS pour stocker les fichiers

- Même si le système de fichiers est stocké sur la même puce flash que le programme, la programmation d'un nouveau ***sketch*** ne modifiera pas le contenu du système de fichiers. Cela permet d'utiliser le système de fichiers pour stocker des données du ***sketch***, des fichiers de configuration ou du contenu pour un serveur Web.



Fonctions du système de fichier SPIFFS

File system object (SPIFFS)

begin

SPIFFS.begin()

This method mounts SPIFFS file system. It must be called before any other FS APIs are used. Returns true if file system was mounted successfully, false otherwise.

format

SPIFFS.format()

Formats the file system. May be called either before or after calling begin. Returns true if formatting was successful.

open

SPIFFS.open(path, mode)

Opens a file. path should be an absolute path starting with a slash (e.g. /dir/filename.txt). mode is a string specifying access mode. It can be one of “r”, “w”, “a”, “r+”, “w+”, “a+”. Meaning of these modes is the same as for fopen C function.

Returns File object. To check whether the file was opened successfully, use the boolean operator.

```
File f = SPIFFS.open("/samplefile.txt", "w"); if (!f) { Serial.println("file open failed"); }
```

Fonctions du système de fichier SPIFFS

File system object (SPIFFS)

exists

SPIFFS.exists(path)

Returns true if a file with given path exists, false otherwise.

openDir

SPIFFS.openDir(path)

Opens a directory given its absolute path. Returns a Dir object

remove

SPIFFS.remove(path)

Deletes the file given its absolute path. Returns true if file was deleted successfully.

.

Fonctions de l'objet SPIFFS

rename

SPIFFS.rename(pathFrom, pathTo)

Renames file from pathFrom to pathTo. Paths must be absolute. Returns true if file was renamed successfully.

info

```
FSInfo fs_info; SPIFFS.info(fs_info);
```

Fills FSInfo structure with information about the file system. Returns true is successful, false otherwise.

seek

file.seek(offset, mode)

This function behaves like fseek C function. Depending on the value of mode, it moves current position in a file as follows:

Fonctions de l'objet SPIFFS

seek

file.seek(offset, mode)

This function behaves like fseek C function. Depending on the value of mode, it moves current position in a file as follows:

- if mode is SeekSet, position is set to offset bytes from the beginning.
- if mode is SeekCur, current position is moved by offset bytes.
- if mode is SeekEnd, position is set to offset bytes from the end of the file.
- Returns true if position was set successfully.

position

file.position(); //Returns the current position inside the file, in bytes.

size

file.size(); //Returns file size, in bytes.

name

String name = file.name(); //Returns file name, as const char*. Convert it to String for storage.

close

file.close();

Close the file. No other operations should be performed on File object after close function was called

Programme Exemple SPIFFS

```
#include <SPIFFS.h>
#include <FS.h>   //Include File System Headers

const char* filename = "/fitest.txt";


void setup() {
  delay(1000);
  Serial.begin(115200);
  Serial.println();

  //Initialize File System
  if(SPIFFS.begin())
  {
    Serial.println("SPIFFS Initialisé....ok");
  }
  else
  {
    Serial.println(" Initialisation SPIFFS...Echec");
  }

  //Format File System
  if(SPIFFS.format())
  {
    Serial.println("Système de fichier formaté");
  }
  else
  {
    Serial.println("Erreur lors du formatage ");
  }
```

Programme Exemple SPIFFS

```
  //Créer un nouveau fichier et y écrire
  //w=Write ouvrir fichier pour écriture
  File f = SPIFFS.open(filename, "w");

  if (!f) {
    Serial.println("Echec ouverture fichier");
  }
  else
  {
      //Write data to file
      Serial.println("Ecritue de données dans le fichier ");
      f.print("Cette phrase est écrite dans  le fichier");
      f.close();  //fermer le fichier
    }

}
```

Programme Exemple SPIFFS

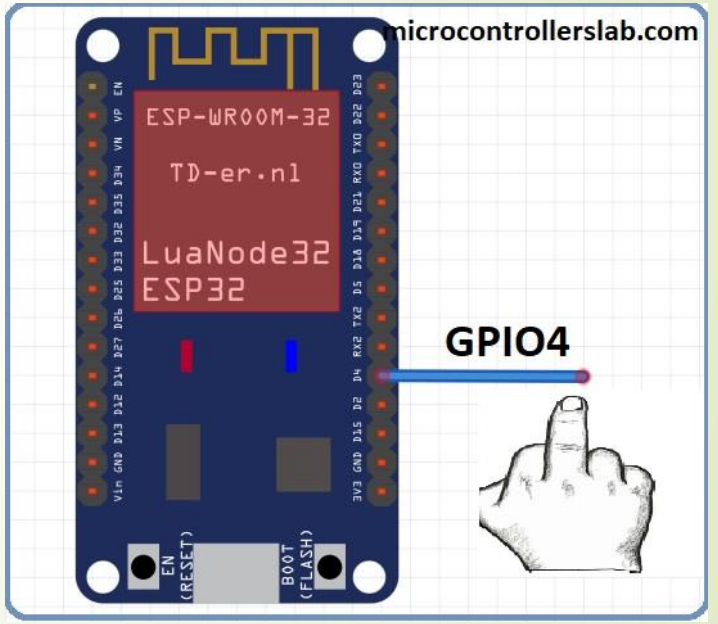
```
void loop() {
  int i;

  //lecture du fichier de données
  File f = SPIFFS.open(filename, "r");

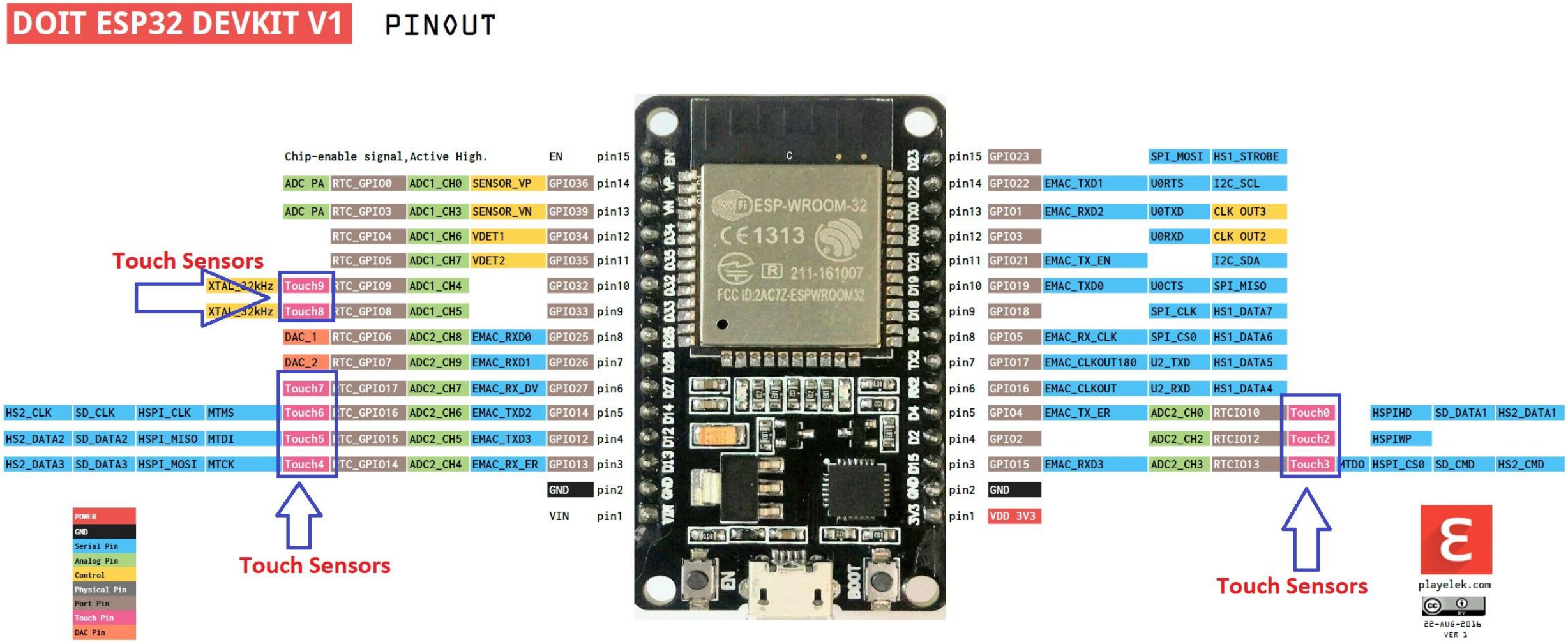
  if (!f) {
    Serial.println("Echec lors de l'ouverture du fichier");
  }
  else
  {
      Serial.println("Lecture du fichier:");
      //récupérer les données du fichier
      for(i=0;i<f.size();i++) // lire tous les octets du fichier
      {
        Serial.print((char)f.read());
      }
      f.close();  //fermer le fichier
      Serial.println("Fichier fermé");
  }
  delay(5000);
}
```

Capteur de toucher ESP32

- ESP32 a des broches qui peuvent détecter le toucher
- **Touch0 >> GPIO4**
- **Touch1 >> pas dispo sur version Devkit 30 pin version mais disponible sur Devkit 36 pin**
- **Touch2 >> GPIO2**
- **Touch3 >> GPIO15**
- **Touch4 >> GPIO13**
- **Touch5 >> GPIO12**
- **Touch6 >> GPIO14**
- **Touch7 >> GPIO27**
- **Touch8 >> GPIO33**
- **Touch9 >> GPIO32**
-



Brochage ESP32 DEVKIT V1



Code

- Exemple de code pour utiliser le capteur de toucher

- EN l'absebce de toucher, la valeur lue est élevée >60
- Lorsque le fil est touchée, la valeur baisse à moins de 30

```
int touch_sensor_value=0;
void setup()
{
Serial.begin(115200);
Serial.println("Exemple Capteur de
toucher ESP32");
}

void loop()
{

touch_sensor_value = touchRead(T4);
Serial.print("valeur Touch4 est = ");
Serial.println( touch_sensor_value);
delay(1000);
}
```

Serveur WebSocket avec ESP32

- ESP32 WebSocket Server: sorties de contrôle (Arduino IDE)
- Dans ce didacticiel, vous apprendrez à créer un serveur Web avec l'ESP32 à l'aide du protocole de communication WebSocket.
- À titre d'exemple, nous allons vous montrer comment créer une page Web pour contrôler les sorties ESP32 à distance.
  - L'état de sortie s'affiche sur la page Web et se met à jour automatiquement dans tous les clients.